

IT Department

Cybersecurity

Team members:

Santiago Arias Gutierrez 2023016

Isaac Campos Gomez 2023033

Joaquín Chinchilla Bonilla 2023042

Jazmin Montero Naranjo 2023090

Isaac Piedra Chaves 2017441

Jimena Valverde Rodriguez 2020479

Jorge Ignacio Ramirez Brenes

Francisco Alvarado Alvarado

Expotec

2026

Summary

Smart home IoT devices, such as cameras, voice assistants, and smart TVs, usually share a network with personal computers, with weak credentials and no updates, which makes them a frequent entry point for botnets and lateral movement. This project presents Quillnet, a security gateway with a Zero Trust architecture built on a Raspberry Pi 4. The gateway segments the home network into four VLANs (users, cameras, voice assistants, and Smart TVs), applies a default-deny firewall with nftables, and inspects traffic inline with Suricata as an intrusion prevention system.

A backend built with Flask and PostgreSQL translates alerts into plain language using an artificial intelligence model, without sending it IP addresses, and displays them in a web application with push notifications. From the application, the user can block devices; in addition, the system applies automatic quarantine after three real blocks within five minutes, unlocking requires two-step verification, and login is protected with a code sent by email.

The research combined a survey of 112 Costa Rican households with security tests launched from Kali Linux and automated backend tests. As of November 5, 2026, the inline blocking of a real beacon from the Covenant framework was verified, as was the fact that unencrypted MQTT traffic to a public broker did not leave the network. It was also found that traffic within the same VLAN does not pass through the gateway and is therefore not inspected, a limitation that was added to the improvement plan together with the remaining pending tests.

Keywords: IoT, Zero Trust, network segmentation, VLAN, IPS, Suricata, Raspberry Pi, smart home.

Table of contents:

Summary....1
Chapter I....4
Project title....4
General Objective....4
Specific Objectives....4
Justification....5
Problem definition....6
Analysis of the problem....7
Problem statement and research question....7
Research objectives....8
Chapter II....8
Theoretical framework....8
Introduction to IoT....8
Updated statistics on the IoT problem....11
IoT protocols and specific vulnerabilities....11
Zero Trust model....12
Network segmentation....12
Purple Team....13
Market solutions....13
Documented real cases....14
Chapter III....15
Methodological framework....15
Type of research....15
Research design....18
Technical concepts of the system....20
Justification of the technical stack....23
Layered architecture....23
Data collection instruments and techniques....37
General procedure....38
Actual execution schedule....39
Scope and limitations....39
Visual design and communication....41
Administrative and financial management....43
Connection and process infrastructure diagram....50
Private network....51
Public network....53
Security analysis....53
1. Project overview....53
2. Scope of the security analysis....54
3. Asset identification....54
4. CIA triad analysis....55
5. Vulnerability identification....58
6. Threat identification....60
7. Risk assessment....60

8. Security measures..... 62
9. Access security and user control..... 63
10. Data protection and privacy..... 65
11. Secure development..... 66
12. Backups..... 67
13. Incident response..... 68
14. Results analysis..... 69
Evidence images..... 73
15. Improvement plan..... 77
16. Conclusions..... 78
Chapter IV..... 79
General Conclusions..... 87
Recommendations for the project's current state..... 88
Recommendations for future development..... 89
Bibliography..... 91
Annexes..... 96

Chapter I

Project title

QUILLNET

General Objective

To implement a physical security gateway, based on a Raspberry Pi 4 and designed to be deployable from the infrastructure of an internet service provider, that applies Zero Trust segmentation through VLANs and a firewall with a default-deny policy, detects and blocks malicious traffic from residential IoT devices inline (IPS mode), and is validated through Purple Team exercises, delivering to the end user a self-management web platform that requires no technical knowledge.

Specific Objectives

1. To segment the home network under the Zero Trust model, using four 802.1Q VLANs (Users, IP Cameras, Voice Assistants, and Smart TVs), with independent DHCP/DNS per segment and an nftables firewall with a forward-drop policy, so that no IoT device can communicate with another segment without an explicit rule.
2. To implement an inline intrusion detection and prevention system, using Suricata 7 in IPS mode (NFQUEUE) with the Emerging Threats Open rules, monitoring services on the Raspberry Pi, and automatic device quarantine, validating its operation through controlled blocking tests and the Purple Team methodology.
3. To build a self-management platform for the end user (backend with a REST API and a web frontend) with a network map, an alert center whose alerts are translated into plain language using artificial intelligence, device blocking and unlocking, and secure authentication, so that the user understands and controls the protection of their home without technical assistance.

Justification

The exponential growth of IoT devices in homes has transformed everyday life, but it has also created an unprecedented attack vector. According to Transforma Insights (2026), around 33.7 billion IoT connections are projected worldwide by 2030, which turns the average home into a vast and heterogeneous digital ecosystem. At the same time, the Netgear and Bitdefender report (2025) estimates that an average home has 22 connected devices and receives 29 attacks per day, which positions IoT devices as one of the main attack vectors in home environments. However, most of these devices lack basic protection mechanisms: Palo Alto Networks' Unit 42 IoT Threat Report (2020) documented that 57% of IoT devices were vulnerable to medium- or high-severity attacks and that 98% of IoT traffic was not encrypted, exposing sensitive data during transmission; although this figure dates from 2020, it illustrates a structural weakness that the more recent data in this chapter do not contradict. This lack of encryption and segmentation means that, once a device is compromised, the attacker can move freely across the entire home network, accessing sensitive information such as banking credentials, personal communications, or security camera images. In other words, the digital home has become a space without perimeters, where privacy is exposed by default.

A critical aggravating factor is that end users lack the technical knowledge needed to manage the security of their IoT networks. Activities such as configuring firewalls, segmenting the network, or monitoring traffic require skills that are beyond the profile of the average resident. This knowledge gap has serious consequences: most households do not apply basic protection measures, such as changing default passwords or enabling encryption, which leaves their devices and data exposed. This situation is not a minor problem of missing updates, but a structural problem that affects millions of homes. Security can no longer depend on individual initiative, because the cost of a mistake is the total exposure of people's private lives. It is therefore imperative to shift the responsibility for security to a higher level: that of the internet service provider (ISP), which already manages the home's connectivity and can, from that position, offer protection without requiring any technical action from the user.

For all of the above reasons, the Quillnet project is justified on three fundamental pillars. The first is the relevance of the problem: the sheer number of connected devices, their high rate of

vulnerability, and the exposure of sensitive data show a real and widespread threat that requires immediate attention, which leads to the second pillar, the urgency to act: users lack the technical training to protect themselves, and the solutions available on the market are not designed to be deployed from the ISP or to offer comprehensive protection that includes attack simulation, segmentation, and intelligent alerts, leaving a functional gap that this project seeks to fill. The third pillar is the feasibility and timeliness of the implementation: the ISP has the infrastructure, the reach, and the strategic position to deploy this system as a managed service, without the end user needing to perform any configuration. This project does not merely analyze the problem; it delivers an operational solution: designing, building, and deploying a working system that integrates Zero Trust segmentation and attack simulations from a network that simulates the ISP, thus protecting homes at scale and in an automated way. In a world where the home is increasingly smart, security can no longer be a luxury or an individual responsibility: it must be a basic service guaranteed from the network.

In the IoT world, security solutions for devices already exist, such as Enterprise IoT Security, which combines automatic device identification through machine learning with Zero Trust policies enforced on next-generation firewalls. Cisco proposes a platform built on TrustSec and Cyber Vision. Fortinet offers FortiNAC, a network access controller that supports multiple vendors and applies dynamic segmentation with automatic quarantine. Zscaler implements a cloud-native model with its Zero Trust Exchange. As can be seen, solutions from large companies exist, but their main limitation is that they are very expensive and not very friendly to inexperienced users. This opens a large security gap for the home sector, and home solutions are uncommon or have been discontinued. That is why Quillnet opens a window of opportunity to protect home networks.

Problem definition

IoT devices in the home (IP cameras, voice assistants, smart TVs) are usually connected to the same flat network as users' personal computers, with no isolation between them, no traffic inspection, and, frequently, with default credentials, outdated firmware, and unencrypted communications. When one of these devices is compromised, the attacker can move laterally to the

rest of the network, exfiltrate private information, or use the device as part of a botnet. The average user cannot detect or resolve this situation alone, and there is no accessible home solution that combines Zero Trust segmentation, intrusion prevention, and an interface that non-technical people can understand.

Analysis of the problem

Technical causes. Flat networks with no segmentation; default passwords (35% of devices keep “admin” as the username, WifiTalents, 2026); CVE vulnerabilities left unpatched for more than two years on 60% of devices (JumpCloud, 2025); 75% with no automatic updates and unencrypted traffic.

Human causes. Lack of knowledge and of a cybersecurity culture; protection depends on individual initiative, and configuring firewalls and VLANs exceeds the skills of the average user.

Market causes. Enterprise solutions are expensive and complex; home solutions are limited, discontinued, or do not apply strict segmentation.

Effects. Lateral movement by the attacker, theft of credentials and camera images, espionage, involuntary participation of the home in botnets and DDoS attacks (Aisuru, BadBox 2.0), and exposure of family privacy.

Affected parties. Homes with IoT devices, in particular users without technical knowledge; indirectly, internet providers and the network infrastructure.

Identified gap. There is no accessible home solution that integrates Zero Trust segmentation, inline IPS, automatic response, and plain-language alert explanations.

Problem statement and research question

Is it feasible to design, from the infrastructure of an internet service provider, a security system based on the principles of Zero Trust segmentation and collaborative Purple Teaming exercises, that significantly reduces the risk of unauthorized access to data generated and transmitted by residential IoT devices, and what are the technical and operational conditions that determine its effectiveness?

Research objectives

1. To identify the main threats to IoT devices in the home environment that can affect users' private information, by collecting threat intelligence data, in order to understand the threats that internet-connected devices face at home.
2. To list the main attack vectors that affect Internet of Things devices in homes, through research on reliable IoT and cybersecurity information sites.
3. To define the attack surface, or the points of vulnerability, in the internet-connected devices that are least protected, in order to determine the degree of protection these devices have.
4. To select the most secure protection systems for protecting IoT networks that can maintain control of traffic and prevent common attacks on these devices.
5. To evaluate, through controlled tests, the effectiveness of the implemented solution in detecting, blocking, and explaining threats, and to identify its limitations.

Chapter II

Theoretical framework

Introduction to IoT

Technology has revolutionized human life, and its achievements allow us to guarantee that things work better when connected to the internet; these devices, which connect to a virtual cloud and communicate with each other, are known as IoT. According to Microsoft Azure, the Internet of Things (IoT) is a network of physical devices, such as cameras, cars, and hospital beds, equipped with sensors, software, and internet connectivity that allow them to detect and exchange data. IoT devices collect, transmit, and process data in real time to help organizations improve operational efficiency, decision-making, and the customer experience in sectors such as healthcare, manufacturing, and transportation.

IoT devices follow a series of protocols that deserve to be analyzed in order to understand how they work. IoT devices rest on four essential pillars, as McRae explains in his article:

Device. The first of the four pillars of IoT is the device. This device can be anything, from a phone to medical equipment used in hospitals, cars, and other appliances. It is any hardware device capable of transferring data from one point to another over the internet.

Data. One of the main goals and purposes of IoT is data collection and storage. This processed data will help improve the application and functionality of different devices and software.

Analytics. It allows each piece of collected data to be analyzed efficiently and accurately, turning the device into a powerful and valuable tool in anyone's life.

Connectivity. The last of the four pillars, which allows the other three to work in harmony. Uninterrupted connectivity is essential to keep real-time data processing and analysis flowing.

This system allows everyday objects to connect to the internet and improve their efficiency. Although these devices arrived to make users' lives easier, they carry a great risk: if one of them is compromised by a malicious actor, the user's private information is exposed, which poses a cyberthreat that, if left unaddressed, could end up harming businesses and homes alike.

Security risks and attacks

IoT security risks stem from many technical vulnerabilities in the hardware, software, and network communications of the devices. These include poor or weak authentication methods, cleartext data transmission, the use of outdated firmware versions, and the exposure of unnecessary network services. Vulnerable devices also offer the possibility of exploiting default passwords, open ports, and the lack of timely software security updates, which creates an entry point for attackers (SentinelOne, 2025).

Among the most common attack vectors against IoT devices are:

Distributed denial-of-service (DDoS) attacks. An attempt to make a device or network inaccessible to its legitimate users by flooding it with traffic or information requests. Hijacked IoT devices can be used to overload servers and cause service outages.

Man-in-the-middle attacks. An attacker intercepts the communication between two devices or networks and impersonates one of them to access confidential information.

Password attacks. Attempts to gain access to a device or network by guessing or cracking the password through brute force.

Malware. Software designed to damage or disable a device or network; IoT devices are particularly vulnerable because they usually have less security than traditional devices.

Physical attacks. Direct tampering with IoT devices that are poorly protected against tampering, allowing an attacker to bypass security measures and gain access to data or control of the device.

Malicious firmware updates. Replacing legitimate firmware with malicious code that can be used to take control of the device or steal stored data.

Insecure wireless connections. Connections that, because they are not properly protected, can be exploited to gain access to the network or to the devices connected to it.

Social engineering attacks. They exploit human weaknesses rather than technical vulnerabilities, by impersonating a legitimate entity to gain access to sensitive information.

Eavesdropping attacks. Interception of communications between devices to obtain confidential information, such as passwords or credit card numbers (Ip_Admin, 2024).

The ports that carry the information of these devices are normally 8883 (secure MQTT), 5671/5672 (AMQP/S), and the web ports 443 (HTTPS/TLS) and 80 (HTTP). As might be expected, the attack surface has also expanded with IoT, so it is essential to develop and follow security best practices to protect devices, users, and the network from cyberthreats (Fortinet, n.d.).

IoT attack surface

According to IBM, an attack surface is the sum of the vulnerabilities, pathways, or methods (sometimes called attack vectors) that attackers can use to gain unauthorized access to the network or to sensitive data, or to carry out a cyberattack. IoT devices, being so vulnerable, have a large attack surface that covers many of the vectors through which they can be breached. According to ManageEngine, these vectors include: connected hardware and devices; software and applications; digital entry points (open ports, running services, public interfaces); users and credentials; and network and connections (VPNs, Wi-Fi, misconfigured firewalls).

Many IoT devices are not securely configured by default when delivered by manufacturers and, owing to their often embedded nature, they are not patched or protected periodically once in production (Jouly, n.d.). A critical step in protecting IoT devices is to harden them through endpoint

protection, closing vulnerabilities in high-risk ports (TCP, UDP), wireless connections, and unencrypted communications. A secure web gateway (SWG), which includes application control, deep HTTPS/SSL inspection, and URL filtering, is also vital. Network security begins with the configuration of a secure firewall, complemented by multi-factor authentication (MFA) and continuous monitoring of network activity. Encryption, both symmetric and asymmetric, is essential to protect data in transit between devices or to the internet (Fortinet, n.d.).

Updated statistics on the IoT problem

Updating the picture of the security problem in IoT devices, more recent information confirms and worsens the diagnosis presented: an average of 22 IoT devices per household is estimated (Netgear/Bitdefender, 2025), with 29 attacks per day recorded per household, a figure that is triple the one reported in 2024 (Bitdefender, 2025). Globally, IoT Analytics (2025) reports a base of 21.1 billion connected IoT devices worldwide at the end of 2025 (+14% compared with 2024) and projects about 39 billion by 2030 (a figure that differs from that of Transforma Insights because of differences in methodology).

Other relevant data show that 35% of IoT devices still use “admin” as the default username (WifiTalents, 2026), that 60% have known vulnerabilities (CVEs) left unpatched for more than two years (JumpCloud, 2025), and that 75% lack automatic updates (WifiTalents, 2026). As for large-scale incidents, Cloudflare (2026) reported that in November 2025 it mitigated the largest distributed denial-of-service (DDoS) attack recorded up to that point, with a peak of 31.4 terabits per second, associated with the Aisuru and Kimwolf botnets, made up largely of compromised home devices, mainly Android TVs.

IoT protocols and specific vulnerabilities

In addition to the general attack vectors described above, it is worth detailing the specific vulnerabilities of the communication protocols most widely used by IoT devices in the home environment.

MQTT (Message Queuing Telemetry Transport). Used by voice assistants and smart sensors, it has no authentication by default and no access control over its communication channels

(topics), which allows unauthorized subscription attacks, command injection, and denial of service against the message broker.

Zigbee. Used in smart light bulbs, locks, and motion sensors, it transmits the network key unencrypted during device pairing, which allows an attacker to capture that key and later decrypt all of the device's communications.

BLE (Bluetooth Low Energy). Used in wearables, smart locks, and medical devices, it has known vulnerabilities in its pairing mechanisms, including BIAS (Bluetooth Impersonation Attacks) attacks.

CoAP (Constrained Application Protocol). Used in resource-constrained sensors, it runs over stateless UDP and has no native authentication in its basic implementations, making it vulnerable to man-in-the-middle attacks, amplification denial of service, and command spoofing.

Zero Trust model

The Zero Trust model is a cybersecurity paradigm based on the principle that no user, device, or service should be trusted by default, regardless of whether it is inside or outside the network perimeter. Every access request must be authenticated, authorized, and continuously verified (NIST, 2020).

Applied to the IoT context, Zero Trust means that every device connected to the home must be treated as potentially insecure. Companies such as Palo Alto Networks have developed solutions that apply this paradigm through the dynamic identification of devices from multiple behavioral attributes, generating automated security policies (Palo Alto Networks, 2024). This approach is particularly well suited to environments where devices do not support the installation of their own security agents.

Network segmentation

Another fundamental method for protecting IoT devices is segmentation. According to Cisco (2026), segmentation is the practice of dividing a network into smaller parts called segments, in order to improve security, performance, manageability, and regulatory compliance. Its effectiveness

lies in isolating networks and devices into independent logical segments, and it is widely used in laboratories to simulate attacks and defenses.

As Bitdefender (2025) points out, segmenting the network consists of dividing it into isolated subnets, so that devices within a segment can communicate with each other, but interactions with other segments are strictly controlled or prohibited. This ensures that, in the event of a security breach in one area, not all devices are compromised at the same time, limiting the attacker's lateral movement. Firewalls play a fundamental role in this strategy, acting as a buffer that separates the network from external threats, blocking malicious traffic while allowing the legitimate exchange of data, and for more advanced protection, the use of firewalls with deep packet inspection is recommended (Nozomi Networks, 2025).

Purple Team

In cybersecurity, traditional teams are divided into the Red Team (offensive, simulates attacks) and the Blue Team (defensive, detects and responds). The Purple Team approach integrates both functions into a collaborative process in which the findings of simulated attacks directly feed the improvement of defenses (MITRE, 2024).

This methodology is particularly useful in IoT security projects, since it makes it possible to empirically validate that protection measures work in real scenarios. The MITRE organization maintains the ATT&CK framework, a structured knowledge base of adversary tactics and techniques that is widely adopted by the industry. Tools such as MITRE Caldera make it possible to automate attack campaigns mapped to this framework, running them against controlled environments to measure the detection capability of the defenses (MITRE Caldera, 2024).

Market solutions

The main companies in the enterprise cybersecurity sector have developed specific solutions to protect IoT devices. Palo Alto Networks offers Enterprise IoT Security, which combines automatic device identification through machine learning with Zero Trust policies enforced on next-generation firewalls (Palo Alto Networks, 2024). Cisco proposes an architecture based on the network as a platform, through TrustSec and Cyber Vision (Cisco, 2024). Fortinet offers FortiNAC, a network access controller that supports multiple vendors and applies dynamic segmentation with automatic

quarantine in response to anomalies (Fortinet, 2024). Zscaler implements a cloud-native model with its Zero Trust Exchange (Zscaler, 2024), and Akamai, through Guardicore Segmentation, specializes in pure microsegmentation of internal lateral traffic (Akamai, 2026).

For the home setting, the options are significantly more limited. Products such as Bitdefender BOX (discontinued in 2021), Firewalls Purple/Gold, Fingbox, or the security features built into routers from NETGEAR (Armor), TP-Link (HomeShield), and ASUS (AiProtection) represent the closest attempts aimed at consumers, although none offers integrated attack simulation or dynamic segmentation at the level of enterprise solutions (NETGEAR, 2025).

This gap between the sophistication of enterprise solutions and the simplicity of consumer solutions is precisely the space that this project seeks to address, translating validated technical principles to the context of the smart home.

Documented real cases

In order to empirically support the relevance of the problem addressed by Quillnet, three recent real cases of security compromises in home IoT devices are documented. Aisuru-Kimwolf botnets (2025): Cloudflare reported the mitigation, in November 2025, of a 31.4-terabit-per-second DDoS attack, the largest recorded up to that point, associated with botnets made up largely of compromised home devices, mainly Android TVs (Cloudflare, 2026). BadBox 2.0 (2025): in June 2025 the FBI warned that this botnet had compromised millions of connected devices (such as TV streaming devices, digital projectors, digital picture frames, and vehicle infotainment systems), many of them inside homes and operating without their owners' knowledge (FBI IC3, 2025).

CVE-2023-1389 (TP-Link): an unauthenticated command injection vulnerability (CVSS 8.8) in the web interface of TP-Link Archer AX21 routers, publicly exploited since April 2023 by a variant of the Mirai botnet and later added to CISA's Known Exploited Vulnerabilities (KEV) catalog (Trend Micro, 2023; CISA, n.d.).

Chapter III

Methodological framework

Type of research

The Quillnet project falls within applied research, a type of research characterized by using existing scientific and technical knowledge to solve concrete problems, as opposed to basic research, whose aim is to generate new knowledge without a defined immediate application. In the case of Quillnet, the team did not start from the need to discover new cybersecurity principles, but from the need to apply principles already established in the industry (network segmentation through VLANs, zero trust architecture, and offensive-defensive validation methodologies known as Purple Team) to a context that, until now, had no accessible solution: the Costa Rican home with internet-connected devices.

This applied orientation rests on a prior diagnosis, developed during the initial phase of the project, in which the team investigated the landscape of cybersecurity solutions already available on the market. That diagnosis made it possible to identify that the existing enterprise solutions, offered by companies such as Palo Alto Networks, Fortinet, Check Point, WatchGuard, Zscaler, Cisco, and Akamai, although robust and effective, are designed for corporate environments, require specialized technical staff to configure and maintain, and have a cost that makes them inaccessible to the average home. On the other hand, the home solutions currently available, such as routers with a basic firewall from brands like TP-Link or ASUS, offer some level of protection, but do not implement strict segmentation policies or a security model based on zero trust. It is precisely in this gap identified between the two ends of the market that Quillnet makes its contribution: bringing enterprise-level security principles to a device designed to be installed and understood by a user without advanced technical knowledge.

Project approach

Given the multidimensional nature of the project, which combines the development of network infrastructure, experimental validation through penetration testing, software development, and the management of a product conceived as a service, Quillnet cannot be analyzed under a single methodological approach. For this reason, the project adopts a mixed-methods approach, which integrates the qualitative and quantitative approaches in a complementary way, each applied to different dimensions of the development.

The qualitative approach is present in two dimensions of the project. On the one hand, in design decisions, user perception, and the interpretation of needs, such as the construction of the visual identity, the color palette, and the user experience of each screen. On the other hand, under a documentary and descriptive design applied to the construction of the theoretical framework: using reliable threat intelligence sources, the relevant concepts of IoT and the types of attack that can compromise it are described, thus grounding the problem that Quillnet seeks to solve.

The quantitative approach, for its part, is present in those dimensions of the project that can be measured objectively and numerically. Examples of this dimension are the logs generated by the Suricata intrusion detection system, the results obtained during penetration tests (for example, whether a simulated attack was detected and blocked or not), the system's response times to an alert, and the financial data associated with the cost of the prototype and its eventual business model.

Justification of the combined approach

The decision to adopt a combined methodological approach, rather than a single one, is because the development of Quillnet integrates three processes of clearly different nature, which occur simultaneously and not in a mutually exclusive sequence.

First, the project contains an applied research component, present from its initial phase, in which both the existing market solutions and the communication protocols used by IoT devices (including MQTT, Zigbee, Bluetooth Low Energy, and CoAP) were studied, with the aim of understanding their characteristic vulnerabilities and technically substantiating the need for the project.

Second, the project contains an iterative development component, since the system was not built all at once, but through successive phases, each subject to review, error correction, and

adjustment based on the results obtained in the previous phase. It is important to clarify that, although the project's academic defenses (first, second, and third defense) are formal evaluation milestones and act as time markers for the expected progress, the team's internal technical work does not depend exclusively on those dates: development continues steadily between one defense and the next, and technical blockers, such as the one the team faced with installing the intrusion detection system on OpenWRT (resolved by migrating to Raspberry Pi OS Lite), are documented and resolved continuously, regardless of the defense calendar.

Third, the project contains an experimental design component, particularly visible in the security validation phase using the Purple Team methodology. This methodology consists of setting up controlled attack scenarios against the system itself, running them, observing whether the implemented defense measures manage to detect and neutralize them, and adjusting the system configuration based on the results obtained. This component of the project is methodologically similar to a classic experimental design, in which there is a manipulated variable (the type of simulated attack) and an observed response variable (whether the detection system manages to identify and block that attack).

The combination of these three methodological components allows the project to be evaluated not only by the final product delivered, but by the soundness, coherence, and traceability of the process that led to that product, a criterion that is central within Expotec's evaluation parameters, where the team is expected to be able to explain and defend the reasoning behind each decision made.

Research design

Project phases

The development of Quillnet was organized into five consecutive phases, each with particular objectives and expected results defined in advance by the team.

The first phase, called the research and definition phase, took place between February and March 2026, and is completed. During this period, the team focused on precisely defining the problem the project would seek to solve: the lack of accessible cybersecurity solutions for IoT devices in the home. As part of this phase, a study of the full landscape of enterprise cybersecurity solutions available on the market was carried out, in order to understand which technologies existed and where the gap that the project could fill lay. In parallel with this technical research, the team worked on building the project's identity, defining the name Quillnet, the mascot Quill, a hedgehog whose natural defense mechanism was adopted as the central metaphor of the project's security approach, and the slogan that represents its working philosophy. Likewise, in this phase the initial five-layer architecture of the system was established and the technical foundation related to IoT communication protocols and the Zero Trust security model was researched.

The second phase, called the documentation and first defense phase, took place between April and May 2026, and is completed. In this phase the project's initial written report was prepared and the team was readied for its presentation to the evaluation panel, achieving approval of the first defense in May 2026. Within this phase, a methodological decision of great importance for the direction of the project was made: the pivot from a virtual laboratory approach, in which the system would depend only on virtual machines and simulated environments, to a real physical device approach. This decision was based on the view that a physical prototype, running on real hardware and real IoT devices, is a more compelling and credible demonstration of the system's functionality before an evaluation panel than a fully virtual simulation.

The third phase, called the system construction phase, took place between June and July 2026 and is completed. During this phase the initial hardware was configured, the access point used to distribute the wireless network was selected, and it was found that OpenWRT version 25.12, by using the apk package manager, prevented the installation of Suricata. In response, the team

replaced OpenWRT with Raspberry Pi OS Lite (Debian), which made it possible to complete the installation of the intrusion detection and prevention system.

The fourth phase, called the backend and frontend phase, took place between August and September 2026 and is completed. In it, the backend (Flask and PostgreSQL, deployed on Render), the gateway services, the web frontend, and the activation of Suricata in IPS mode were built, backed by automated backend tests (238 tests as of the November 4, 2026 cutoff). Its intermediate evaluation milestone was the project's second defense, in August 2026.

The fifth phase, called the polish and Expotec phase, takes place between October and November 2026 and is in progress. It comprises the final refinement of the system across all of its layers, ahead of its public presentation. Its first milestone was the project's third defense, held between October 12 and 16, 2026. Since November 1, 2026, the code has been frozen (only critical fixes are accepted) in order to prioritize stability, and the phase closes with the final presentation at Expotec, on November 13 to 15, 2026.

It is worth clarifying, as part of the methodological design of this research, that the dates of the academic defenses do not strictly determine the pace of the project's technical progress. These defenses are formal milestones of external evaluation, but the team keeps working steadily between one and the next, adjusting its work rhythm to each member's availability and to the complexity of the technical obstacles that arise during development.

Population and context of application

Quillnet is designed to operate within the context of the Costa Rican home, specifically in homes that have internet-connected IoT devices, such as security cameras, voice assistants, and smart TVs. The user profile for which the system is intended is a person without specialized technical knowledge of networks or cybersecurity, able to install and use the system without constant external technical assistance.

The prototype developed for its presentation at Expotec 2026 is a working small-scale proof of concept. This prototype simulates the conditions of a real home network by connecting test IoT devices through the access point configured by the team, making it possible to demonstrate how the

segmentation, detection, and threat response system works in an environment that is controlled but representative of the real conditions of use of the product.

Technical concepts of the system

To understand the internal operation of the Quillnet system, it is necessary to first define some fundamental technical concepts on which its architecture rests.

API (application programming interface). A set of rules and functions that allows different software components to communicate with each other: it receives a request, passes it to the corresponding component, and returns a response. Quillnet uses APIs with distinct responsibilities: one developed in Python that receives the logs generated by Suricata and sends them to the backend, and an external artificial intelligence API that interprets that technical information and turns it into an explanation understandable to the homeowner.

Flask. A lightweight framework for developing web applications with Python, which provides the tools needed to create routes, forms, sessions, and database connections (Pallets Projects, 2025). In Quillnet it is used to develop the backend (a REST API with 27 routes), served with Gunicorn on Render, which connects the frontend with the PostgreSQL database, the AI alert translation service, and the commands to the gateway.

Suricata. An open-source cybersecurity tool that can work simultaneously as an intrusion detection system, an intrusion prevention system, and a network monitoring tool. It analyzes network traffic and uses rules to identify suspicious events, generating alerts and logs in JSON format through its EVE system.

Two-factor authentication (2FA). A security mechanism that requires two different identity verification methods belonging to different categories. In Quillnet, login requires an email, a password, and a 6-digit code sent by email, and unlocking a device additionally requires a temporary code sent by email, so that an open session is not enough to remove a protection.

Backend and frontend. The frontend is the visual part of the application that the user interacts with directly. The backend is the internal part in charge of receiving the frontend’s requests, applying the system logic, querying the database, and communicating with the other components. Although both run on the same machine during the prototype stage, they are logically distinct components.

Server. A machine, program, or combination of both that provides information or functions to a client. In the final version of Quillnet the backend runs on a cloud server (Render), while the gateway (Raspberry Pi) acts as a client: it always initiates the connection to the server, because it sits behind NAT and cannot receive incoming connections.

PostgreSQL. An open-source relational database management system (PostgreSQL Global Development Group, 2025). In Quillnet it replaced SQLite, used in the initial local prototype, and stores accounts, devices, alerts, and the translations generated by the artificial intelligence, with each household's data kept separate by means of a gateway identifier (multi-tenant architecture).

VLAN and IEEE 802.1Q. A VLAN (Virtual Local Area Network) is a logical broadcast domain created on the same physical infrastructure; the IEEE 802.1Q standard defines the tagging of Ethernet frames with a VLAN identifier (IEEE, 2022). In Quillnet it separates devices into four segments.

nftables. The Linux kernel's packet filtering framework, successor to iptables, which organizes rules into tables and chains with default policies (Netfilter Project, n.d.). In Quillnet it is used with a forward-drop policy: everything that is not explicitly allowed is denied.

NAT, DHCP, DNS, and dnsmasq. NAT translates private addresses to a public address in order to reach the Internet (Srisuresh & Egevang, 2001); private addresses follow RFC 1918 (Rekhter et al., 1996). DHCP automatically assigns IP addresses to clients (Droms, 1997) and DNS resolves domain names; dnsmasq provides both services with an independent scope per VLAN (Kelley, n.d.).

IPS and NFQUEUE. An IDS detects and alerts; an IPS also blocks malicious traffic inline. NFQUEUE is a Linux kernel queue that hands packets to a user-space program, such as Suricata, to decide whether they are accepted or dropped (Netfilter Project, n.d.; Open Information Security Foundation, 2025). The Emerging Threats Open rules are community threat signatures (Proofpoint, n.d.). Bypass mode (fail-open) keeps traffic flowing if the engine stops responding, prioritizing availability.

JWT and multi-tenant. A JSON Web Token is a standard for signed tokens used to transmit identity and permissions between parties (Jones et al., 2015). In Quillnet every request carries the gateway identifier, so a single system serves several households with isolated data (multi-tenant).

scrypt and rate limiting. scrypt is a memory-hard key derivation function, designed to make brute force harder against stored passwords (Percival & Josefsson, 2016). Rate limiting restricts the number of requests per interval to mitigate repeated access attempts. Quillnet stores passwords with scrypt (the werkzeug.security library, parameters scrypt:32768:8:1) and a random salt per password. Verification codes are stored as HMAC-SHA256, a hash-based message authentication code with a secret server key.

SSE (Server-Sent Events). An HTTP mechanism by which the server sends events to the browser in real time (WHATWG, n.d.); Quillnet uses it to display alerts instantly.

Web Push and PWA. Web Push is the standard that allows a server to send notifications to a person's device even when the page is closed, using VAPID keys and a service worker. A PWA (progressive web app) is a web page that is installed as an application. Quillnet implements both; on iPhone, the notification requires installing the PWA from the browser.

Language model (LLM) and Groq. An LLM is an artificial intelligence model trained to understand and generate text. Quillnet uses Llama 3.3 70B (Meta, 2024) through the Groq API (Groq, n.d.) to translate technical alerts into plain language. Only the rule signature, its category, the action, and the protocol are sent to the model; never IP addresses, MAC addresses, or device names. If the primary model fails, the request is retried with a fallback model (llama-3.1-8b-instant) and, if that also fails, the alert is saved with fallback text according to its severity.

Raspberry Pi 4 Model B. A low-cost single-board computer with a quad-core ARM processor and a Gigabit Ethernet port (Raspberry Pi Ltd, 2025), sufficient to run the operating system, the firewall, and Suricata simultaneously.

Justification of the technical stack

The selection of each technology within the Quillnet architecture was based on specific technical criteria, not on an arbitrary choice.

Suricata versus Snort. Suricata was selected as the intrusion detection system, instead of Snort, mainly because Suricata is multi-threaded, which allows it to take advantage of the four processing cores of the Raspberry Pi 4, whereas Snort runs its analysis on a single thread, making it slower under the traffic of multiple devices. In addition, Suricata has better native support for modern IoT protocols, is compatible with existing Snort rules, and supports an active prevention mode.

Raspberry Pi OS Lite versus OpenWRT. OpenWRT was initially selected as the network operating system because it is one of the most stable options for ARM architecture, because of its support for VLANs under the 802.1Q standard, and because it includes a built-in nftables firewall. However, its version 25.12 introduced the apk package manager and did not allow Suricata to be installed, so it was replaced with the 64-bit Raspberry Pi OS Lite (Debian 13), which offers greater compatibility with Suricata, Python, and the rest of the project's technologies, as well as extensive documentation for configuring router, switch, and gateway functions.

VLANs versus firewall rules alone. It was decided to complement the firewall rules with segmentation through VLANs because these create isolation at the network level: a device connected to a given VLAN cannot see the traffic belonging to another VLAN. This is a physical-logical isolation barrier, additional to and complementary with the nftables firewall rules applied on the gateway.

Layered architecture

Physical layer

The physical layer of the system consists of a Raspberry Pi 4 Model B, with 4 gigabytes of RAM, which works as the central gateway of the entire home network. The selection of this hardware was based on specific criteria of cost, availability in the local market, and sufficient processing capacity to simultaneously run the network operating system, the firewall, and the intrusion detection system without compromising the overall performance of the device.

Within this layer, the gateway receives Internet through its wireless interface (wlan0), connected to a phone hotspot that stands in for the internet provider during the demonstration, and delivers the segmented network through its native Ethernet port (eth0) to the TP-Link EAP110 access point, over a link with 802.1Q tags. This distinction is of particular methodological importance, since it guarantees that all of the home's traffic, both incoming and outgoing, must necessarily pass through the Raspberry Pi before reaching any device, which is the physical basis that makes it possible to apply the segmentation and Zero Trust security policies.

Segmentation and detection layer

On top of the physical layer described above, the 64-bit Raspberry Pi OS Lite (Debian 13) was installed as the gateway system, with nftables as the firewall, NetworkManager for the VLANs, dnsmasq for DHCP/DNS, and Suricata as the intrusion prevention system. OpenWRT had been tried previously and was discarded for the reasons set out in the justification of the technical stack.

Installation and configuration process, documented step by step:

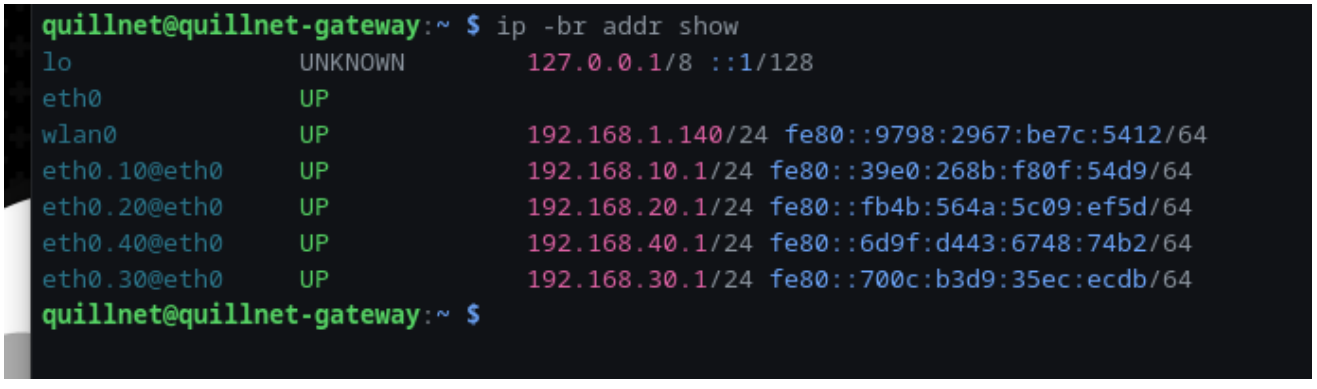
The 64-bit Raspberry Pi OS Lite was installed on the Raspberry Pi 4's microSD card and the system was updated, leaving the gateway ready for network configuration.

Internet access was configured through the wlan0 wireless interface, connected to the phone's hotspot, and packet forwarding with network address translation (NAT) toward that interface was enabled.

The TP-Link EAP110 access point was configured as the wireless distribution point, connected to the Raspberry Pi's Ethernet port through a link with 802.1Q tags, publishing one SSID for each VLAN.

With basic connectivity established, the VLANs were created with NetworkManager (nmcli) and each was assigned its own DHCP/DNS service with dnsmasq, in order to separate the different types of devices on the network, distributed as follows:

- VLAN 10: Users (192.168.10.0/24)
- VLAN 20: IP cameras (192.168.20.0/24)
- VLAN 30: Voice assistants (192.168.30.0/24)
- VLAN 40: Smart TVs (192.168.40.0/24)



(Source: The team's own work)

The TP-Link EAP110 access point (hardware v4.0, 2022 firmware 5.0.4) publishes four SSIDs, one per VLAN: Quillnet-Users (VLAN 10), Quillnet-Cameras (VLAN 20),

Quillnet-SmartHomes (VLAN 30, voice assistants) and Quillnet-TV (VLAN 40). All four networks use WPA2-PSK with AES encryption and a different password per VLAN, so that knowing the key of one network does not give access to the others; the guest network is disabled and the radio operates on 2.4 GHz (channel 11, 802.11 b/g/n). The AP is managed on VLAN 10 (192.168.10.119). Its LAN port negotiates at 100 Mbps full duplex, so the 802.1Q trunk link between the Raspberry Pi and the AP is limited to Fast Ethernet.



2.4GHz SSIDs

Add

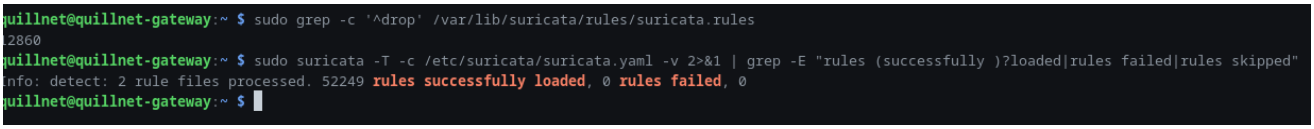
ID	SSID	VLAN ID	SSID Broadcast	Security Mode	Guest Network	Action
1	Quillnet-Users	10	Enable	WPA-Personal	Disable	
2	Quillnet-Cameras	20	Enable	WPA-Personal	Disable	
3	Quillnet-SmartHomes	30	Enable	WPA-Personal	Disable	
4	Quillnet-TV	40	Enable	WPA-Personal	Disable	

(Source: The team's own work)

Once the VLANs were created, the nftables firewall was configured under the Zero Trust model, with a forward-drop policy: only explicitly authorized traffic is allowed. First, all VLANs were allowed to access the internet, so that the devices would work normally. Then, access from the camera, voice assistant, and Smart TV VLANs to the users' VLAN was blocked, so that, if any of those devices were ever compromised, it could not reach the network where the users' computers are. The users' VLAN was also allowed to communicate with the others in order to manage the IoT devices when necessary. Finally, communication among the IoT device VLANs was blocked, so that each one remained isolated from the others.

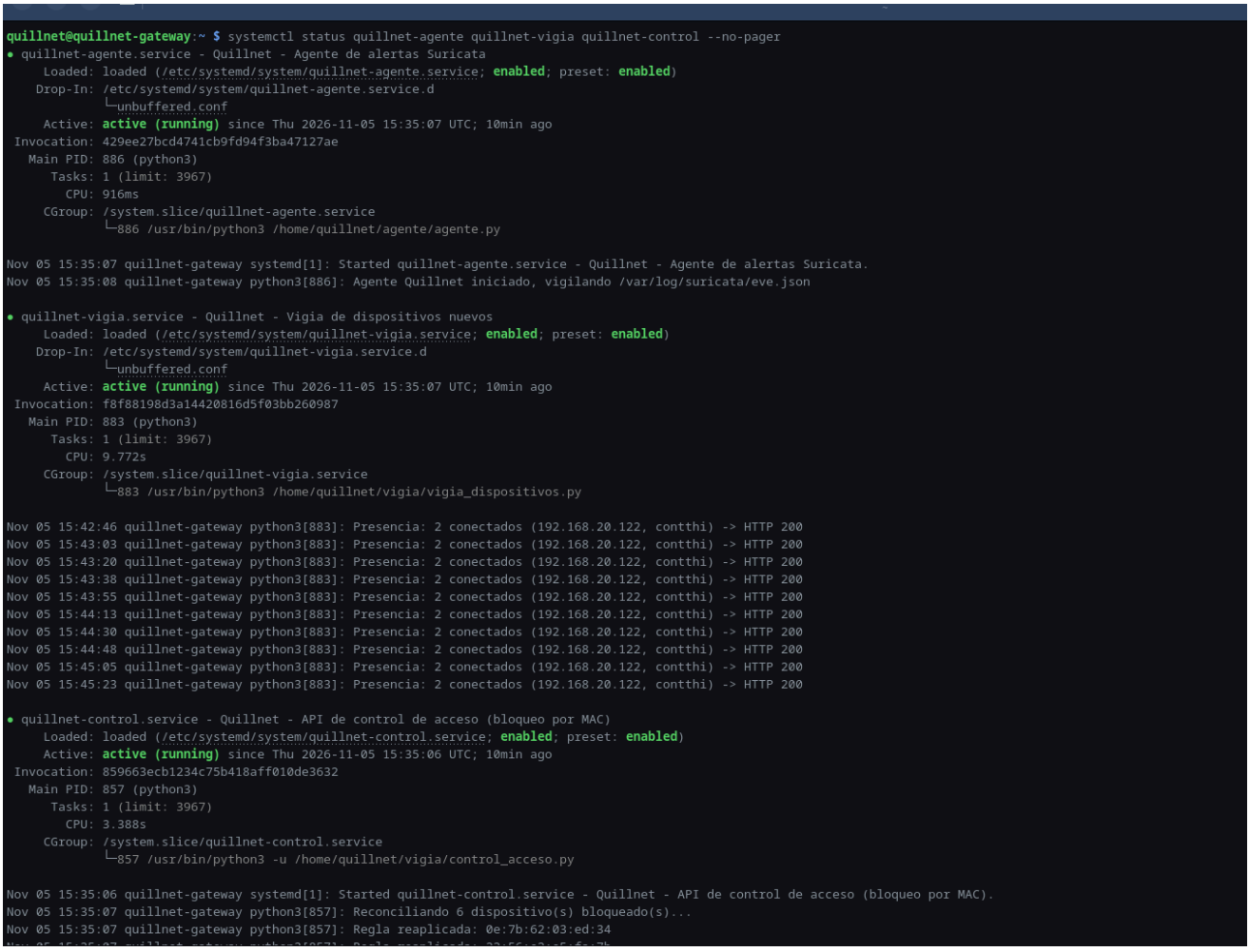
As part of this same layer, Suricata 7.0.10 was installed in inline IPS mode, integrated with nftables through NFQUEUE and with bypass (fail-open) so as not to leave the home without Internet if the engine fails. It has 52,246 rules loaded (Emerging Threats Open and the team's own demonstration rules), none of them failed, of which 12,860 operate in drop mode, that is, they actually block traffic. The custom rules are five: 9000001 (sqlmap tool), 9000002 (/quillnet-ataque path), 9000010 (remote control), 9000011 (suspicious traffic), and 9000020 (unencrypted MQTT).

By difference, 52,241 rules come from Emerging Threats Open and 5 are custom. The technical obstacle initially identified with OpenWRT was resolved by migrating to Raspberry Pi OS Lite.



(Source: The team's own work)

Suricata is started through a systemd override file (override.conf) with the -q 0 option, which connects it to kernel queue NFQUEUE 0; this enables it as an inline IPS (it can drop packets) and not just as a passive IDS. The gateway's three own services (quillnet-agente, quillnet-vigia, and quillnet-control) are systemd units with automatic restart (Restart=always). quillnet-control runs as root because it needs to manipulate nftables, while quillnet-agente and quillnet-vigia run under the unprivileged quillnet user, in application of the principle of least privilege.



(Source: The team's own work)

Validation and pentesting layer

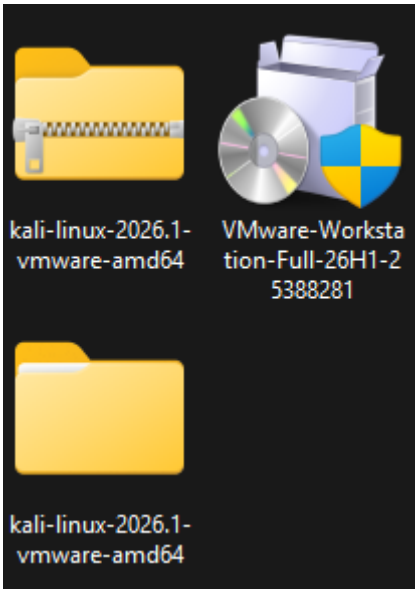
The validation of the security measures implemented in the previous layers is carried out by applying the Purple Team methodology, a cybersecurity approach that combines the offensive perspective of a Red Team, in charge of simulating real attacks against the system, with the defensive perspective of a Blue Team, in charge of detecting and responding to those attacks.

Technology selection. To run the Kali Linux virtual machine, two desktop hypervisors were initially evaluated: VirtualBox and VMware Workstation Pro; VMware was used for its higher performance and lower overhead. The team later migrated the test environment to KVM/QEMU managed with virt-manager on Fedora Linux, with Kali Linux 2026.2, as this is virtualization built into the Linux kernel and open source . As for the testing operating system, Kali Linux was chosen over other alternatives because it is the industry standard for ethical hacking and pentesting, and among its greatest advantages is that it is a fairly complete toolbox that comes already configured.

Installation and configuration process, documented week by week:

First, the original VMware installer for 64-bit Windows was downloaded, together with the Kali Linux virtual machine folder, and VMware Workstation was installed.

Note: the following screenshots correspond to the initial stage of the laboratory with VMware Workstation on Windows; the current environment runs on KVM/QEMU (virt-manager) on Fedora.

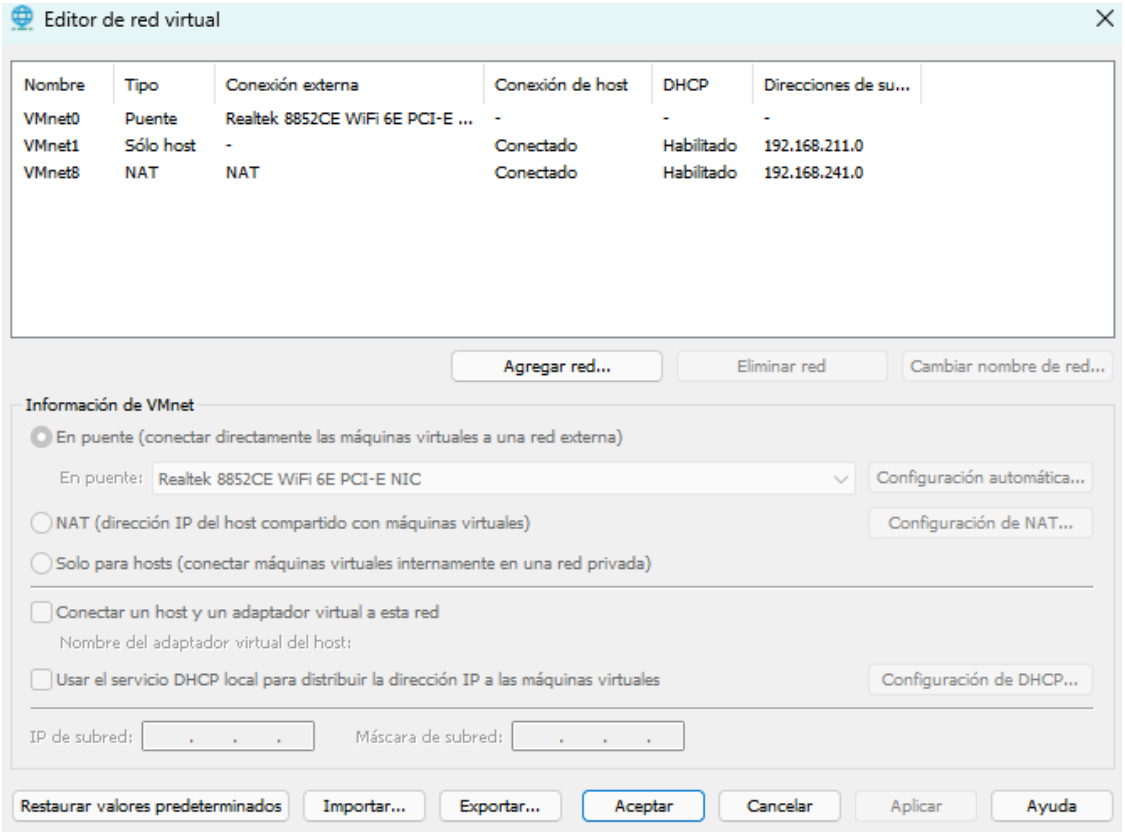


(Source: The team's own work)

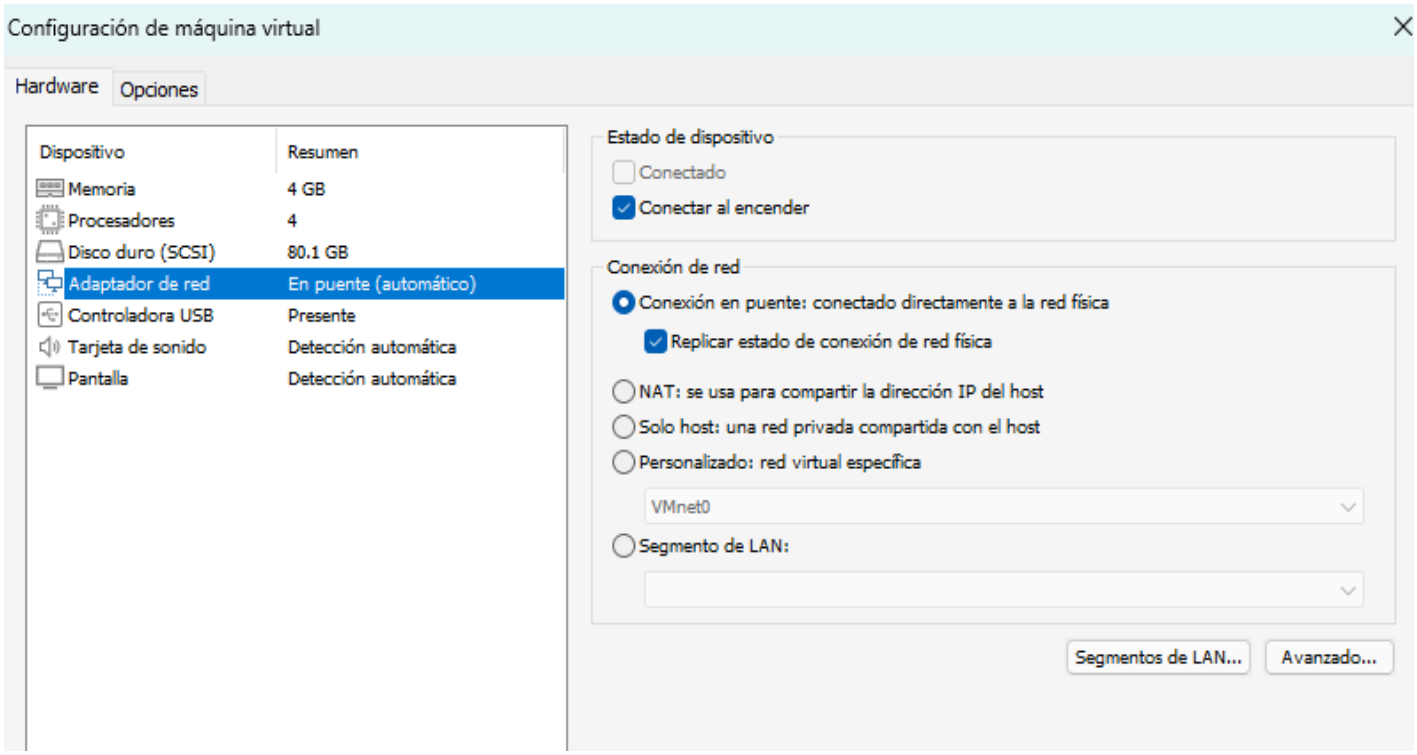
Afterwards, a stage was devoted to installing the virtual machine inside VMware Workstation and adjusting the resources it consumes.



(Source: The team's own work)

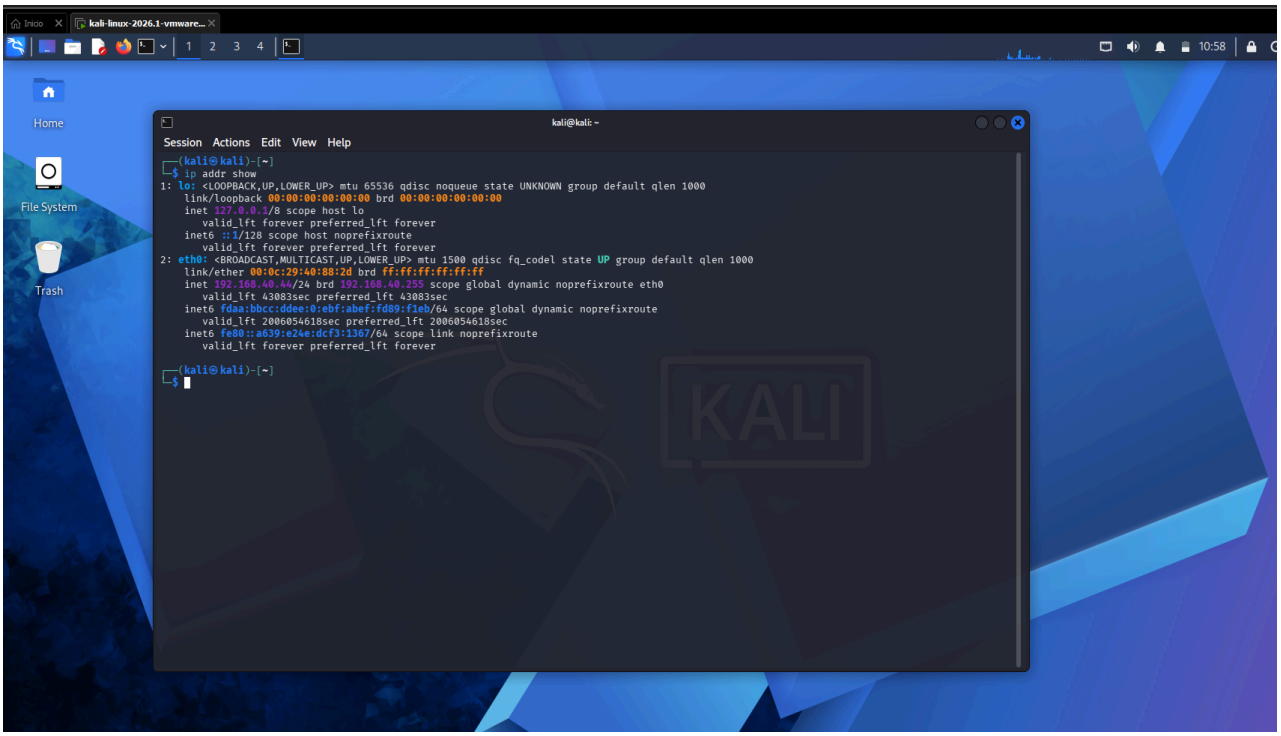


(Source: The team's own work)

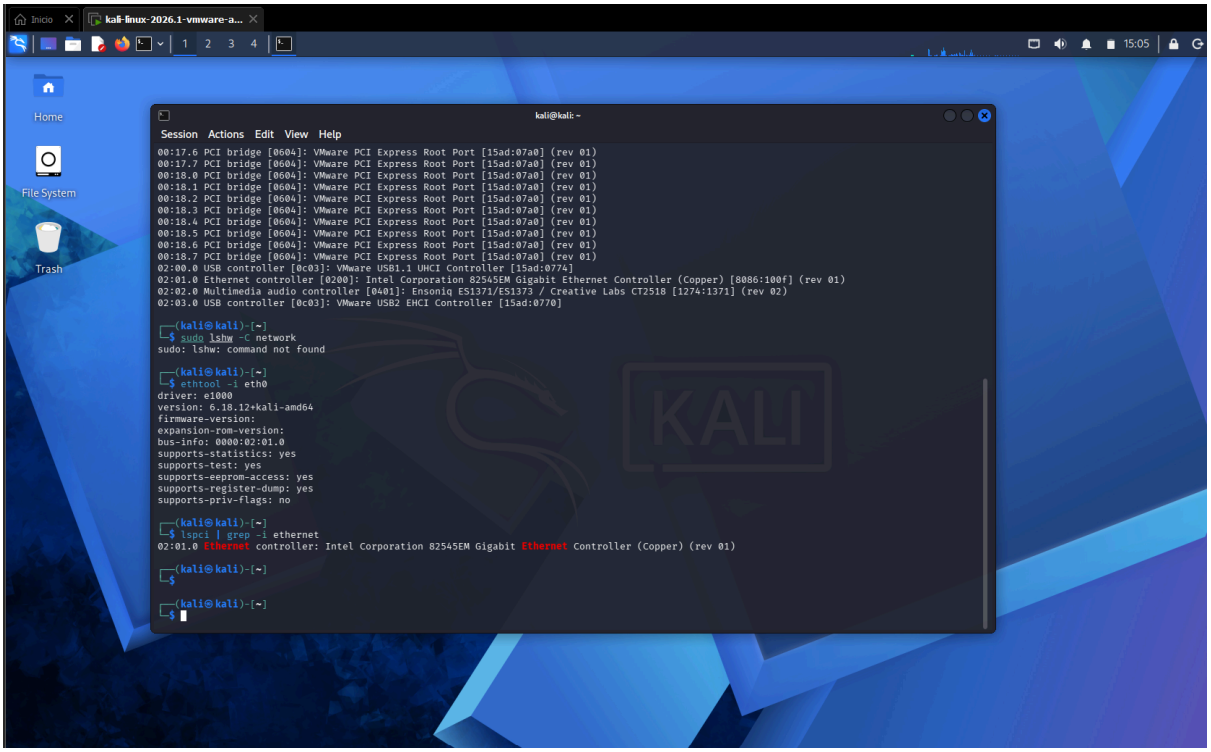


(Source: The team's own work)

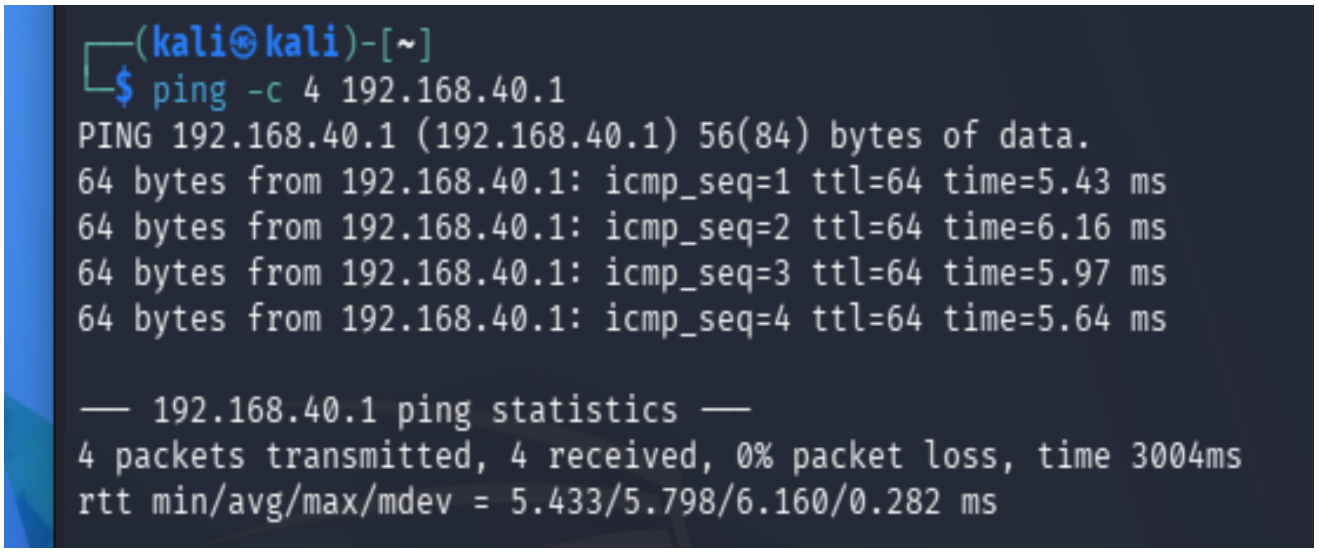
Complementing this configuration, and so that Kali Linux would appear as an independent device on the network, it was configured and ensured that the interface used was specifically the laptop's WiFi interface.



(Source: The team's own work)



(Source: The team's own work)



(Source: The team's own work)

As of the writing of this document, the test environment is complete and manual detection and blocking tests were run (Feodo Tracker, the NSIS_Inetc signature, trojan check-in, MAC address blocking, and persistence after reboot and power cut), described in the security analysis. The formal Kali test battery for each VLAN, verification of detection on VLANs 20, 30, and 40, the VLAN hopping test, and the performance test under simultaneous load remain pending. Verification cutoff: November 5, 2026. The manual tests, all launched from VLAN 10 with a Kali machine (192.168.10.171), demonstrate the inline blocking of a real beacon from the Covenant framework (signature ET MALWARE Covenant Framework HTTP Beacon, sid 2027793) and that unencrypted MQTT traffic to a public broker did not leave the network; they do not replace the formal pentesting battery.

Backend layer

The backend of the Quillnet system is made up of three elements that operate in a coordinated way. The gateway's Python services (quillnet-agente, quillnet-vigia, and quillnet-control) read the Suricata logs, monitor which devices are actually connected (every 15 seconds), and every 5 seconds check for pending commands to run them with nftables. A Flask 3.1 server with Gunicorn, deployed on Render, exposes 27 routes. A PostgreSQL database (10 tables and 13 Alembic migrations) stores the information, isolated per household by means of a gateway identifier included in the JWT. The Raspberry Pi always initiates the connection to the server, because it sits behind NAT.

Database construction process with alert translation, documented during its development:

The first step consisted of creating a database with an integrated artificial intelligence translator. Tests were run with files in JSON format: if an alert's identifier is not registered in the database, the system sends the full text of the file to the Groq API, which translates it into simple language; that translated text is shown to the user and, in addition, the identifier is stored together with the translation in the database. In this way, if the same alert occurs again, the system simply looks up the identifier that already exists in the database, instead of querying the artificial intelligence again.

At a later stage, the database was migrated from SQLite to PostgreSQL, its schema was versioned through migrations, and progress was made on defining the functional requirements of the complete system.

As of the writing of this document, the backend, the database with the artificial intelligence translator (model llama-3.3-70b-versatile, without sending IP addresses), automatic quarantine (3 blocks in 5 minutes), email delivery, and real-time events are implemented and tested by means of 238 automated backend tests (pytest), run against a disposable local PostgreSQL database and never against production, with email and DNS simulated. Push notifications (Web Push) are implemented in the backend and covered by automated tests.

Alert classification by attack type. Each alert is classified into one of 13 types: Malware, Remote control, Dangerous connection, Network scan, Brute force, Web attack, Intrusion attempt, Data theft, Denial of service, Cryptomining, Risky browsing, Unknown device, and Suspicious traffic. Fixed, deterministic rules are applied first (the event type, brute-force keywords, the prefix of the Emerging Threats rule such as ET MALWARE or ET SCAN, the Suricata category, and, lastly, keywords); only if those rules do not decide, the artificial intelligence chooses from a closed list, so it cannot invent categories. This decision arose from two earlier versions: one based solely on keywords, where almost everything fell under “Other,” and another where the AI created free-form categories, which scattered equivalent names (for example, “Malware” and “Malicious software”).

Security mechanisms implemented in this layer. (1) The Raspberry Pi generates its own authentication secret (32 random bytes) and delivers it only once at registration; the backend stores only its SHA-256 hash, so it cannot reveal it again. (2) Passwords are stored with an scrypt hash (werkzeug.security, parameters scrypt:32768:8:1, with a random salt per password) and can only be changed through a single-use link sent by email: the token is random, is stored as SHA-256, expires in 30 minutes, and the response is identical whether or not the account exists, to prevent user enumeration. (3) Sessions use JWT with a 7-day validity, and a password change invalidates previous sessions. In addition, login has two-step verification: with a correct email and password the session is not granted, but a 6-digit code arrives by email and the session is granted only when the person enters it. The code expires after 10 minutes, allows 5 attempts, can be used only once, and is stored only as an HMAC-SHA256 with a server key; an unlock code cannot be used to log in and,

with an incorrect password, no email is sent. If the email cannot be sent, the server responds with 502 and access is not granted; the LOGIN_2FA=0 variable on Render turns the code off as an emergency switch. The remaining secrets use SHA-256 (the Pi's secret and the password reset token). (4) Rate limiting exists on login, registration, password recovery, and unlocking. (5) Unlocking a device requires two-step verification (password and a 6-digit code sent by email, protected with HMAC-SHA256, valid for 10 minutes with a maximum of 5 attempts), whereas blocking is a direct action, because blocking reduces risk and unlocking reopens it. (6) Every query is filtered by the gateway identifier (multi-tenant isolation) and a foreign identifier returns 404 so as not to confirm its existence. (7) The backend sends security headers (CSP, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and HSTS) and limits CORS to an explicit list of origins.

Frontend layer

The system's frontend layer is developed as a web application built with HTML, CSS, and JavaScript, without using an additional framework, designed to be accessible from any modern internet browser, without requiring the end user to install additional software.

In its final version, the application includes an isometric network map with one slab per VLAN and Quill the hedgehog at the center; an expanded view per VLAN where devices appear as fireflies with their status written out (New or Alert); a statistics Dashboard; an Alert Center with plain-language explanations; device management (approve, block, and unlock with two-step verification); and Settings. It works as a progressive web application (PWA) with light and dark themes, receives real-time alerts through Server-Sent Events, and meets accessibility criteria such as always showing status in text (not just in color), respecting the reduced-motion preference, and maintaining a minimum contrast of 4.5:1.

Screen definition methodology. Before writing a single line of code, each of the twelve screens that make up the application was defined following the same five-element format: purpose (what problem it solves), user flow (what the user sees and does, in order), contract with the backend (what is requested and what is received), what can go wrong (expected errors), and the why (justification of the technical decisions). This process allowed the team to know exactly what was being built and why, before programming began.

As a representative example of this process, three of the twelve defined screens are documented below:

Login. Its purpose is to confirm that the person in front of the screen is an authorized user of the home. When credentials are incorrect, it was decided to show a generic message (“Incorrect username or password”), without specifying which of the two failed, so as not to confirm to a potential attacker that the user does exist. Password validation lives entirely in the backend, never in the frontend, since browser code can be inspected by anyone.

Main dashboard. Its purpose is to answer, at a glance and without technical jargon, whether everything is fine on the home network at that moment. In the implemented version, the home’s status is calculated with four levels, always accompanied by text and not only by color: “protected”; “needs attention” (there are new alerts or a device has been in quarantine for less than 15 minutes); “at risk” (there is an open high-severity alert); and “offline” (the gateway has gone more than 3 minutes without sending a signal). Since only medium- and high-severity alerts reach the system, normal traffic does not trigger an alarm. The Dashboard also shows the indicators for the last 24 hours, alerts per day over the last week, an attack-type chart, and a matrix of attacks by zone (VLAN) and type.

“Is it you?” verification (2FA on suspicious behavior). Unlike the normal login, this verification is triggered when the system detects unusual behavior in an account that is already logged in. It was established as a rule without exceptions that the session is closed locally immediately when this notice appears, without allowing the logged-in person to keep browsing while deciding whether it was them or not. If they confirm it was them, they must enter an additional code sent by email before opening a new session; if they indicate it was not them, the system blocks access immediately, with no intermediate steps. In the implemented version, this email verification code is required to unlock a device.

The remaining nine screens (Account registration/linking, Network map, New device detected, Device detail, Alert Center, Statistics, and Settings, among others) followed the same documented definition process before their visual development.

The decision to use a physical device responds to better handling of the home’s real devices: a physical device that works hand in hand with the router is more suitable than a virtual laboratory

that is not tangible for the user, and it makes installation easier thanks to its two clearly differentiated interfaces.

The project's entire architecture is built on the rules of Zero Trust, a cybersecurity model that operates under the premise of "never trust, always verify." Quillnet does not trust any of the devices connected to the network, so it segments them into VLANs and prevents their communication from being open. Although this model may seem rigid, it is a viable solution for protecting the networks of IoT devices.

System requirements

As part of the backend and frontend development methodology, the team defined in advance a set of functional and non-functional requirements, with the purpose of guiding the construction of the system before starting its technical implementation.

Functional requirements :

ID	Requirement	Status
FR01	User registration using email, password, and phone number	Met with adjustment: registration asks for a linking code, email, and password; it does not request a phone number; login adds two-step verification by email ("Check your email" screen with countdown and resend)
FR02	Secure login (JWT) and two-step verification (2FA) to unlock devices	Met
FR03	Security analysis API in Python — processes Suricata logs, detects threats, and sends results to the backend	Met
FR04	Network response API — device isolation by VLAN, device blocking by MAC address (nftables), application of security rules	Met
FR05	Generation of automatic reports with analysis results, risks, and recommendations	Partial: the Dashboard offers a summary of network status and 24-hour and 7-day statistics; there is no exportable report
FR06	Threat classification into medium and high levels (low severity is discarded as noise) and into 13 attack types	Met
FR07	User notifications for critical threats or suspicious activity	Partial: real time (SSE) and email ready; push implemented, phone delivery pending testing
FR08	Control panel (dashboard) with network, device, and alert status	Met

ID	Requirement	Status
FR09	Security recommendations (password changes, updates, etc.)	Met
FR10	History of events and security analysis	Met (90-day retention)
FR11	Secure logout, removing active access	Met: Settings > Account includes log out

Non-functional requirements :

ID	Requirement	Status
NFR01	Performance — network analysis must not significantly affect its operation	Pending (no load tests)
NFR02	Usability — intuitive interface that is easy to use for any type of user	Partial: no formal user testing
NFR03	Privacy — user data is not shared without authorization	Met
NFR04	Response time close to real time	Met

Data collection instruments and techniques

Technical instruments

The collection of the project's technical data is carried out mainly through two sources. The first is the logs generated automatically by the Suricata intrusion detection system, which document the network traffic analyzed and the threats identified. The second source corresponds to the results obtained during the execution of the tools used in the pentesting validation phase, which make it possible to determine objectively whether a simulated threat was correctly detected and neutralized by the system.

Documentary instruments

As a complementary instrument for collecting and tracing information, the project keeps a centralized development log on the Notion platform. In this log, each team member documents individually and independently their assigned tasks, their chronological record of activities carried out, the technical problems encountered during their work and the possible solutions identified, as well as their planned next steps. This documentary instrument is of particular methodological relevance, since it makes it possible to reconstruct precisely, at any point in the project, the complete sequence of decisions made, the obstacles faced by each work area, and the solutions applied

throughout the entire development process, thus constituting the main source of evidence for the trial-and-error process documented throughout this chapter.

Validation with external parties

In addition to the technical and documentary instruments described above, the project has an external validation mechanism, provided by the academic advisors designated for the project, who follow the team's progress and give feedback on its technical and documentary decisions at the project's formal milestones. This mechanism is complemented by the team's own verification through controlled tests, particularly valuable when technical blockers arise, as happened with the conflict related to installing the intrusion detection system on the initial version of OpenWRT used in the project. That blocker was resolved by the team by replacing OpenWRT with Raspberry Pi OS Lite.

General procedure

In order to maintain methodological coherence among the project's different work areas, such as the technical, design, and administrative parts, the team applied the following general procedure across the board in the development of each component of the system:

First, the specific requirement or problem to be solved within that particular component of the system is precisely defined. Second, the conceptual design of the proposed solution is carried out, including the description of the expected flow of operation and the early identification of possible points of failure. Third, the technical implementation of the component is carried out, following the previously agreed conceptual design. Fourth, the implemented component is subjected to functional tests under controlled conditions. Fifth, the results obtained during those tests are documented in detail, including any problem or error identified. Sixth, the necessary adjustments or corrections are made based on the documented results. Finally, this complete cycle is repeated for the development of the next component or for the next improvement iteration of the same component.

This procedure was not applied exclusively to the technical development of the system, but was also adopted in the development of the project's visual identity and in its administrative and financial planning, which explains the methodological coherence present in the different work areas described in the following sections of this chapter.

Actual execution schedule

In order to honestly reflect the project's real methodological process, and not only an idealized plan, the project's execution schedule is presented below, contrasting the milestones already achieved with those that are still pending as of the writing of this document.

During May 2026 the first academic defense was approved, closing the second phase. In July 2026 the Suricata installation blocker on OpenWRT was identified, resolved by migrating to Raspberry Pi OS Lite. Between August and September the backend, the frontend, and the IPS were built, and the second defense took place in August.

Looking ahead to what remains, the third defense was held between October 12 and 16, 2026, and the project will be presented to the general public and the evaluation panel as part of Expotec, on November 13 to 15, 2026. The summary of the project's actual execution schedule is presented below:

Phase	Period	Status
Phase 1 — Research and definition	February-March 2026	Completed
Phase 2 — Documentation and first defense	April-May 2026	Completed
Phase 3 — System construction	June-July 2026	Completed
Follow-up meeting	July 2026	Held; blocker resolved with Raspberry Pi OS
Phase 4 — Backend and frontend	August-September 2026	Completed
Second defense	August 10-13, 2026	Held
Phase 5 — Polish and Expotec	October-November 2026	In progress
Third defense	October 12-16, 2026	Held
Final presentation at Expotec	November 13 to 15, 2026	Pending

Scope and limitations

It is important to clearly delimit, within the methodological framework of this project, what the real scope of the developed prototype is and which aspects are explicitly outside that scope.

The prototype developed for its presentation at Expotec 2026 is a working proof of concept, and does not represent a fully finished commercial product ready for mass distribution. Among the elements outside the scope of the current prototype is the implementation of a security information and event management system, known in the industry as SIEM. Although this type of system was considered in the early stages of the project, the team decided to reposition it as a future scalability line of the project, and not as a component that is part of the prototype currently under development, because of the additional complexity its implementation would involve within the available time.

Similarly, it must be clarified that, although the team has used remote access tools as an internal collaboration resource during the construction of the prototype, these tools are not part of the final product's architecture. Quillnet operates with the gateway inside the home's local network and a cloud backend (Render); the gateway always initiates the connection to the backend, and alert translation depends on the Groq API, so it requires an Internet connection for as long as that external dependency remains.

In addition, the prototype has limitations inherent to its stage of development. Its operation depends on external tools and services (Suricata, Flask, PostgreSQL, Render, Groq, and Resend), so any change in their availability may affect it. In particular, login with two-step verification depends on Resend delivering the code: with the test sender (onboarding@resend.dev), messages are delivered only to the email address that owns the Resend account, so before the presentation delivery must be confirmed or the code must be turned off with LOGIN_2FA=0. The prototype is initially oriented toward a single installation, uses a single gateway with no redundancy, and its performance depends on the capacity of the Raspberry Pi. The PostgreSQL database on Render's free plan expired on October 27, 2026; the team renewed it with a paid plan for the database at $6 per month. In addition, the gateway does not yet have an input chain in nftables that protects its own services.

Other known limitations are: the rate limiter keeps its counters in the memory of a single process, so they are not shared; the real-time connection (SSE) sends the token in the URL, which could end up in access logs; sessions are invalidated only by a password change or by expiration, with no explicit revocation; and, until a custom domain is verified in the email service, messages are delivered only to the owner of that service's account. Load tests have not been carried out either.

Inspection limitations identified in the tests: traffic between machines on the same VLAN is switched without passing through the gateway, so Suricata does not inspect it (an sqlmap run launched from the Kali machine against another machine on VLAN 10 did not generate an alert); the IPS also cannot apply content signatures to TLS-encrypted traffic; and low-severity informational signatures are discarded by design to avoid alert fatigue. These limitations justify VLAN segmentation and leave as improvements client isolation on the access point, rules based on TLS metadata (JA3/SNI), and DNS filtering. In addition, the link between the Raspberry Pi and the access point is 100 Mbps, owing to the limitation of the EAP110's port.

Finally, the main methodological limitation identified to date is that the formal experimental validation through pentesting (a Kali battery for each VLAN, detection on VLANs 20, 30, and 40, the VLAN hopping test, and performance under simultaneous load) is still pending. The manual tests carried out demonstrate the blocking of real threats, but they do not replace that validation, whose progress will be documented in later updates to this written report.

Visual design and communication

Brand identity construction process

The visual identity of the Quillnet project was not conceived as an aesthetic element added after the technical development, but as a component defined from the project's initial research phase, following its own methodological process. This process included defining the project name, creating a conceptual mascot, a hedgehog named Quill, whose natural defense mechanism consists of raising its quills only in the face of a real threat, without needing to attack actively, and defining a slogan that synthesizes this philosophy: "Defense that does not need to attack." This metaphor is not merely decorative, but directly reflects the Zero Trust security philosophy applied by the system: a defensive posture, passive by default, that reacts only to an identified threat.

Color palette selection criteria

The definition of the project's color palette followed, like the technical components described earlier in this chapter, a documented methodological process, and not an arbitrary selection. This process began with a research stage on color psychology, aimed at understanding which emotions and perceptions are commonly associated with different color combinations. From that research, two

candidate palette combinations were initially identified, each associated with a different set of values and perceptions that the project could convey to its users and evaluators.

As part of this process, the team explicitly defined which values and perceptions the product should convey to someone seeing it for the first time, including: a sense of trust, calm, elegance, and sophistication, and an image of technological innovation that would be attractive and memorable. Based on these criteria, the following color combination was selected as the project's final palette:

- Black (#000000), used to represent the digital environment and the invisible threats that exist within a home network, while also conveying seriousness and professionalism.
- Navy blue (#000080), used to represent the protection and secure state of the monitored devices, conveying stability and calm to the user.
- Electric purple (#BF00FF, used as a decorative purple, while buttons and interactive elements use #9D00FF), used to represent technological innovation and the intelligence of the system, in direct reference to the fact that the system does not merely protect passively, but analyzes network traffic, detects suspicious behavior, and responds to possible threats. This color also connects conceptually with the Purple Team methodology applied in the project, which combines precisely an offensive and a defensive perspective to strengthen the security of the system.
- White (#FFFFFF), used to represent the clarity and transparency of the information delivered to the user, reflecting the principle that users must be able to understand clearly, through charts and plain-language alerts, what is happening on their own home network.

As for the project's final typography, the Noto Sans Telugu type family was selected, applied hierarchically according to the type of content to be displayed in the interface: the Semibold (600) weight is used for titles, the Regular (400) weight for subtitles, and the Thin (100) weight for the application's general text, in order to establish a clear visual hierarchy among the different levels of information presented to the user. In the frontend implementation, titles, large figures, and the text logo use Rowdies, and the body uses Noto Sans Telugu (Google Fonts, with a system-ui fallback). A Thin (100) weight in running text does not meet the minimum 4.5:1 contrast declared in the frontend layer, so Regular (400) should be used for running text.

Frontend screen definition

As described earlier in the section on the frontend layer within this chapter, the definition of each screen of the application followed a systematically documented process, applied consistently throughout the project. For each screen, its specific purpose, its expected interaction flow, its data exchange contract with the backend, and the possible points of failure that could arise were defined and documented before its technical development. This same methodological process, originally applied to the technical development of the interface, was led in close collaboration with the project's design and communication area, in order to ensure that the functional decisions and the aesthetic decisions for each screen followed the same coherent logic.

Iterative mockup process

The visual development of the system's different screens was carried out following an iterative process, in which the team started from purely conceptual definitions of each screen, described in terms of purpose and functional flow, and progressively advanced toward increasingly elaborate and refined visual representations. This iterative approach allowed visual style adjustments to be made independently, without needing to modify the functional logic of each screen, which had already been agreed upon and documented beforehand by the team.

Administrative and financial management

Distribution of responsibilities and roles

The Quillnet team distributed its responsibilities into six complementary roles, each documented individually in the project log: a leadership and general project coordination role, in charge of strategic decision-making and of overseeing the entire system architecture; a network infrastructure role, in charge of the design, installation, and configuration of the physical network and of the segmentation and detection components; a pentesting role, in charge of penetration testing and of documenting the evidence obtained during those tests; a web development role, in charge of building the system's backend, database, and frontend; a design, marketing, and communication role, in charge of the project's visual identity, the application's graphical interface, and

communication with external parties; and a financial and resource administration role, in charge of the project's overall planning, the assessment of its costs, and its viability as a service.
This distribution of roles allows each member to document their own methodological work process independently, while the leadership role maintains the overall coherence of the project and ensures that the different work areas advance in coordination with one another.

Purchase decision-making process

Decisions related to the acquisition of hardware for the project followed, like the technical and design decisions described earlier in this chapter, a process of evaluating alternatives before the final selection, rather than an arbitrary choice of the first available product.

A concrete example of this process, documented in the project log, is the selection of the access point used to distribute the prototype's wireless network. Initially, the team had considered acquiring the TP-Link EAP615 model; however, that model was not available on the local market at the time its acquisition was required. Faced with this limitation, the team evaluated the available alternatives and finally selected the TP-Link EAP110 model, a decision made considering both the immediate availability of the product and its fit with the overall budget defined for the project.

Budget management

As part of the project's administrative and financial management, a business plan was drawn up that included, among other elements, a market study of existing cybersecurity solutions and a detailed projection of the costs associated with the developed prototype.

The total hardware investment required to develop the prototype amounts to one hundred twenty-seven thousand nine hundred Costa Rican colones, an amount distributed between the purchase of the Raspberry Pi 4 Model B, at a cost of one hundred ten thousand five hundred colones, and the purchase of the TP-Link EAP110 access point, at a cost of seventeen thousand four hundred colones.
As part of the project's scalability projection beyond its stage as an academic prototype, a projected business model was proposed in which an internet service provider (ISP) would offer Quillnet as a service through a monthly subscription of three thousand five hundred colones (₡3,500) per

household. This scenario is hypothetical and does not imply an agreement with any company. The estimated target market is 430,000 people interested in protecting their digital network.

For a new product, a conversion rate of 2.5% of the target market (430,000 people) after one and a half years is projected, which is equivalent to 10,750 connected customers. The curve starts at zero customers and adds users month by month thanks to telemarketing campaigns, the ISP's app, and advertising.This price was contrasted with the willingness to pay observed in the survey (between 5 and 10 additional dollars per month, Chapter IV), which, using a reference exchange rate of ₡453.43 per dollar (date: 11/05/2026), is equivalent to ₡2,267.15 – ₡4,534.30 per month. The projected price of ₡3,500 falls within that range, so the model does not require an ISP subsidy to be acceptable to the surveyed households. This conclusion is limited to the sample of 112 households, which is non-probabilistic.

• Months 1 to 3 (launch and education): slow adoption, with about 300 new customers per month. By the end of the quarter there are 900 customers generating ₡3,150,000 per month.

• Months 4 to 6 (commercial traction): the pace rises to 500 new customers per month. By month 6 there are 2,400 active customers generating ₡8,400,000 per month.

• Months 7 to 12 (product maturity): steady growth of 700 new customers per month. By month 12 there are 6,600 active customers billing ₡23,100,000 per month.

• Months 13 to 18 (consolidation): 700 sign-ups per month continue until the cap of 10,750 customers is reached in month 18, with billing of ₡37,625,000 per month.

Month	Active customers	Monthly revenue	Cumulative revenue
3	900	₡3,150,000	₡6,300,000
6	2,400	₡8,400,000	₡26,250,000
12	6,600	₡23,100,000	₡128,100,000
14	8,000	₡28,000,000	₡181,650,000
18	10,750	₡37,625,000	₡317,975,000

The projected initial investment is ₡172,000,000. With a subscription of ₡3,500 per month, cumulative revenue exceeds that investment in month 14 (₡181,650,000 cumulative; at month 13 it was ₡153,650,000). This timeframe is calculated on revenue, not profit: it does not deduct cloud operating costs, technical support, or maintenance.

Management of relationships with external parties

The management of the project's relationship with its external parties and with the academic advisors designated for the project, is documented centrally in the project log, including the dates of the meetings held, the agreements reached during those meetings, and the next steps defined jointly between the team and those external parties.

This management has been a shared responsibility mainly between the project leadership role and the design, marketing, and communication role, which is also in charge of tasks related to the project's external communication and outreach, thus complementing the relationship with the academic advisors managed directly by the team leadership.

Data collection instruments

The project included a form in which respondents filled in information about their devices and basic information about their households. The questionnaire's questions can be divided into three categories:

General household information: In which province is your home currently located? How many people live in your household? What age range are you in? What is your household's monthly income? The general information serves to obtain data on the type of population the Quillnet project is intended for, obtaining fundamental information about the sample we took.

Information about devices: Number of cell phones, smart assistants (such as speakers), computers, tablets, and smart TVs. This knowledge is relevant for understanding the average number of devices in a home, not only to have real data on the use of IoT devices in homes but also to improve Quillnet to cover a wide range of devices.

Information about user awareness: Are you and your household aware that all of those devices can be, or are being, compromised by an attacker? Did you and your household know that in Costa Rica there are around 255 million attacks on devices like the ones you have at home? If there were a service that could protect your networks from these attacks and give you relevant information about the state of your connections, integrated into your internet service for a small added fee, would you use it? If your previous answer was yes, how much do you consider would be

a good price for that added value? These questions were designed to understand the respondents' level of awareness regarding cybersecurity applied to their devices at home.

Methodological note: the figure of 255 million attacks in Costa Rica mentioned in one of the questionnaire's questions could not be verified in a public source (the team's search did not locate it), so it is not used as data in the theoretical framework or in the conclusions of this work.

1. Devices

N.º	Device	Specification / function	Quantity
1	Raspberry Pi 4 Model B	4 GB of RAM, 4-core ARM. Central gateway: routing, VLANs, firewall, and IPS.	1
2	TP-Link EAP110 access point	Distributes the wireless network with one SSID per VLAN.	1
3	microSD card	Stores Raspberry Pi OS Lite and the gateway configuration.	1
4	USB-C power supply and Ethernet cables	Power and link to the access point.	1 set
5	Mobile phone (hotspot)	Provides the WAN; stands in for the internet provider in the demonstration.	1
6	Test IoT devices	IP camera, voice assistant, and Smart TV, one per VLAN.	3
7	Laptop with Fedora Linux and Kali Linux (virtual machine)	Offensive testing station (Red Team): Kali Linux 2026.2 on VMware) , with Nmap, sqlmap, curl, and tmNIDS.	1

Component	Version / detail	Use in Quillnet
Raspberry Pi OS Lite 64-bit	Debian 13 “trixie”	Gateway operating system
NetworkManager (nmcli), nftables, dnsmasq	Forward-drop policy; one DHCP/DNS scope per VLAN	802.1Q VLANs, Zero Trust firewall, and network services
Suricata	7.0.10, inline IPS (NFQUEUE) with bypass; 52,246 rules loaded, 12,860 in drop mode, 5 custom demonstration rules	Detection and blocking of malicious traffic
Gateway Python services	quillnet-agente v4 (Suricata alerts), quillnet-vigia v4 (presence every 15 s), and quillnet-control (commands every 5 s, MAC blocking)	Event reporting, device monitoring, and command execution
Flask 3.1 + Gunicorn on Render	27 routes	Backend (REST API)
PostgreSQL + Alembic	10 tables, 13 migrations	Multi-tenant database
Groq (llama-3.3-70b-versatile, fallback llama-3.1-8b-instant), Resend, and Web Push (VAPID)	—	Alert translation, email delivery, and push notifications
HTML, CSS, and JavaScript	No frameworks	Frontend: network map, alerts, light/dark theme
Kali Linux, Nmap, sqlmap, curl, tmNIDS, mosquitto-clients, jq, Notion, and Git/GitHub	—	Offensive testing, eve.json analysis, project log, and version control

Software and services

2. Items

Item	Cost (₡)	Notes
Raspberry Pi 4 Model B (4 GB)	110,500	Central gateway
TP-Link EAP110 access point	17,400	Replaced the EAP615, not available on the local market
Total main hardware	127,900	Base investment for the prototype
microSD, power supply, and cables	0	Accessories
Software (Suricata, nftables, Flask, PostgreSQL, Debian)	0	Free and open-source software

Item	Cost (₡)	Notes
Cloud services (Render, Groq, Resend)	$6 per month for the DB.	Render: web service ; Render PostgreSQL database renewed with a paid plan; Groq and Resend on free plans

Description of the solution or prototype

Quillnet is a physical security gateway placed between the Internet and the home network. All device traffic must pass through it, which makes it possible to segment it, inspect it and, if it is malicious, block it before it reaches its destination. The solution is made up of three cooperating parts: the gateway (Raspberry Pi 4 with VLANs, nftables, and Suricata IPS), the cloud backend (Flask and PostgreSQL on Render), and the self-management web frontend.

When Suricata detects or blocks malicious traffic, the quillnet-agente service (which forwards only medium- and high-severity alerts) sends the event to the backend, which stores it, classifies it into one of 13 attack types, translates the alert into plain language with artificial intelligence (without sending IP addresses, MACs, or names), and displays it in real time on the dashboard; it also notifies through Web Push and, when the alert is high severity and there is no active push subscription, by email. It is important to distinguish the cases: a command-and-control beacon such as the Covenant one is blocked inline; the demonstration detection rules (such as the sqlmap one) generate an alert without blocking, and traffic between machines on the same VLAN does not pass through the gateway, so Suricata does not inspect it. If a single device accumulates 3 blocks in 5 minutes, the system automatically places it in quarantine; to unlock it, the owner must confirm with

two-step verification. The Raspberry Pi checks the backend's command queue every 5 seconds and always initiates the connection, so it is not necessary to open ports on the home modem.

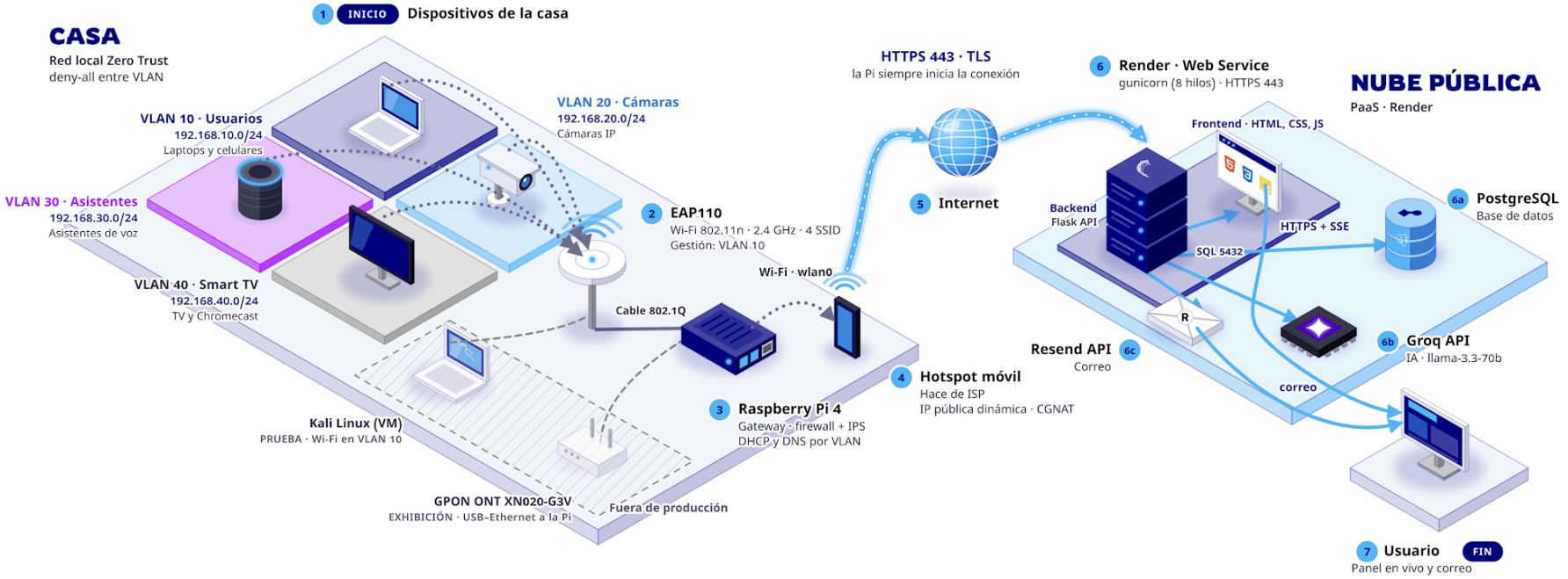
The project's philosophy is represented by its mascot, Quill the hedgehog, who raises its quills only in the face of a real threat: "Defense that does not need to attack." The detail of each layer (physical, segmentation and detection, validation, backend, and frontend) is described in the Layered architecture section.

Connection and process infrastructure diagram

Infraestructura de red · Quillnet

Gateway IoT Zero Trust en Raspberry Pi 4 · del dispositivo de la casa (inicio) al usuario en la nube (fin)

1 Dispositivos › 2 EAP110 › 3 Pi › 4 Hotspot › 5 Internet › 6 Render › 7 Usuario



Direccionamiento IP

SEGMENTO	RED PRIVADA	GATEWAY
VLAN 10 Usuarios	192.168.10.0/24	.10.1
VLAN 20 Cámaras	192.168.20.0/24	.20.1
VLAN 30 Asistentes	192.168.30.0/24	.30.1
VLAN 40 Smart TV	192.168.40.0/24	.40.1

RFC 1918 · DHCP .100–.200 (dnsmasq)

PUNTO	IP PÚBLICA / WAN
wlan0 · Pi	Dinámica, la asigna el hotspot
Hotspot · ISP	Pública dinámica · CGNAT
Render	quillnet.onrender.com

Conexiones

TRAMO	TIPO	PROTOCOLO · VELOCIDAD
1 → 2	Inalámbrica	Wi-Fi 802.11n · ≤ 300 Mbps
2 → 3	Física	Ethernet + 802.1Q · 10/100/1000
3 → 4	Inalámbrica	Wi-Fi 802.11 (wlan0) · variable
4 → 6	Internet	HTTPS / TLS 1.2+ · 443
6 → 6a	Interna	PostgreSQL · TCP 5432
6 → 6b·6c	Internet	HTTPS · API de IA y correo
6 → 7	Internet	HTTPS + SSE · 443
GPON	Fibra	ITU-T G.984 · 2.5/1.25 Gbps

Procesos

3 Raspberry Pi

nftables · firewall Zero Trust
Suricata · IPS · dnsmasq · DHCP + DNS
agente · vigía · control (Python)

6 Render · Flask 3.1 + gunicorn (8 hilos)

/api/alerts · /api/devices/presencia
/api/commands/pending · /ack
/api/gateways/register
6a usuarios, dispositivos, alertas, comandos
6b explica alertas sin datos personales
6c recuperación de contraseña y 2FA

Servicios y puertos

SERVICIO	TRANSPORTE	PUERTO
HTTPS panel + API	TCP · TLS	443
DNS (dnsmasq)	UDP / TCP	53
DHCP (dnsmasq)	UDP	67 / 68
SSH administración	TCP	22
PostgreSQL	TCP	5432
MQTT (previsto)	TCP · TLS opc.	1883 / 8883

Sentido: la Pi siempre inicia (saliente).
Desde Internet no entra nada a la casa.

Leyenda

- Cable (físico)
- Inalámbrico (Wi-Fi)
- HTTPS casa → nube
- Llamadas dentro de la nube
- Fuera de producción

GPON ONT XN020-G3V · exhibición
Sin fibra real · USB-Ethernet a la Pi

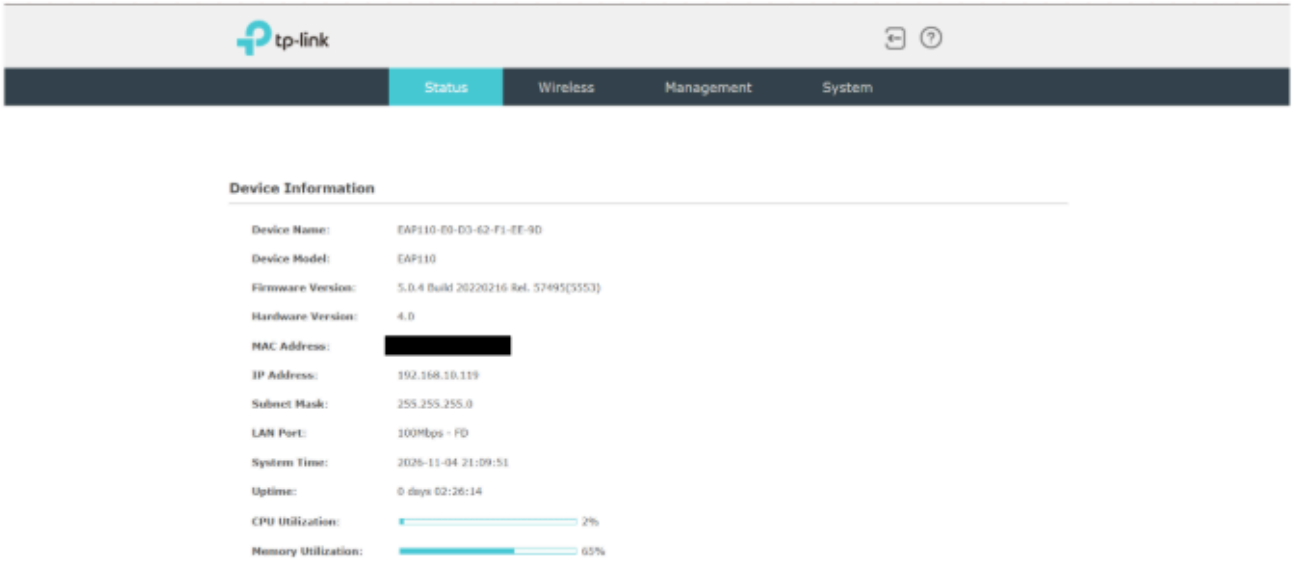
Kali Linux (VM) · prueba
KVM/QEMU bridged · Wi-Fi en VLAN 10

Private network

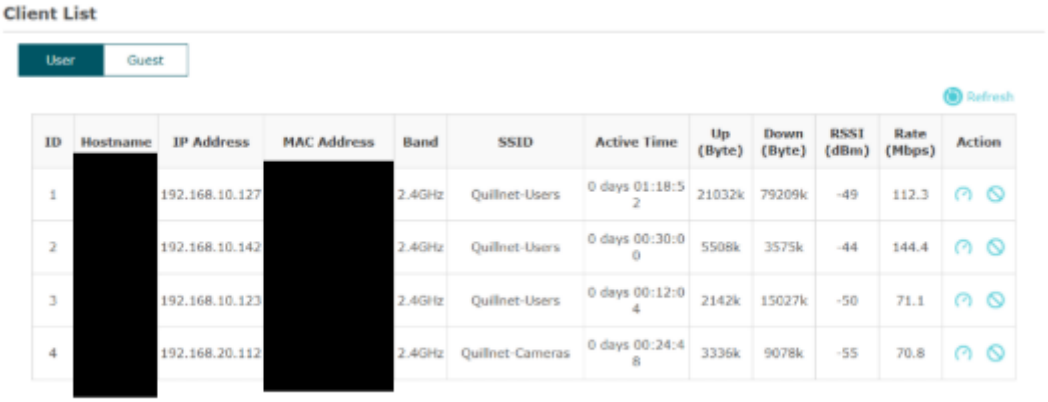
The private network is the home's segmented LAN, made up of four VLANs with addresses from the RFC 1918 private ranges (Rekhter et al., 1996), not routable on the Internet:

VLAN	Segment	Subnet	Devices
10	Users	192.168.10.0/24	Cell phones and computers of the people in the household
20	IP cameras	192.168.20.0/24	Video surveillance cameras
30	Voice assistants	192.168.30.0/24	Smart speakers and assistants
40	Smart TV	192.168.40.0/24	Televisions and set-top boxes

Addressing verified on the gateway: each VLAN's gateway is the .1 address of its subnet (for example, 192.168.10.1 for Users) and dnsmasq hands out addresses from the .100 to .200 range with a 24-hour lease time on each VLAN. The four VLANs travel tagged (802.1Q) over the eth0 interface to the TP-Link EAP110 access point, which connects to VLAN 10 (management) and is excluded from alert and presence reports so as not to generate infrastructure noise.



(Source: The team's own work)



Client List

User | Guest

Refresh

ID	Hostname	IP Address	MAC Address	Band	SSID	Active Time	Up (Byte)	Down (Byte)	RSSI (dBm)	Rate (Mbps)	Action
1	[redacted]	192.168.10.127	[redacted]	2.4GHz	Quillnet-Users	0 days 01:18:52	21032k	79209k	-49	112.3	
2		192.168.10.142		2.4GHz	Quillnet-Users	0 days 00:30:00	5508k	3575k	-44	144.4	
3		192.168.10.123		2.4GHz	Quillnet-Users	0 days 00:12:04	2142k	15027k	-50	71.1	
4		192.168.20.112		2.4GHz	Quillnet-Cameras	0 days 00:24:48	3336k	9078k	-55	70.8	

(Source: The team's own work)

The firewall policy is Zero Trust (forward drop): all VLANs can reach the Internet under Suricata inspection; there are no inter-VLAN rules, so no VLAN can initiate communication toward another: traffic between VLANs is cut off by the firewall without generating a Suricata alert (test report of November 5, 2026).

```
quillnet@quillnet-gateway:~ $ sudo nft list ruleset
table inet filter {
        chain forward {
                type filter hook forward priority filter; policy drop;
                ether saddr 62:87:69:f2:28:40 drop
                ether saddr 5e:6c:f5:65:93:ff drop
                ether saddr c6:09:e8:8b:4d:bf drop
                ether saddr 5a:46:4b:13:5a:8a drop
                ether saddr 22:56:e2:e5:fa:7b drop
                ether saddr 0e:7b:62:03:ed:34 drop
                iifname { "eth0.10", "eth0.20", "eth0.30", "eth0.40" } oifname "wlan0" accept
                ct state established,related accept
        }

        chain IPS {
                type filter hook forward priority filter + 10; policy accept;
                queue flags bypass to 0
        }

        chain postrouting {
                type nat hook postrouting priority srcnat; policy accept;
                oifname "wlan0" masquerade
        }
}
quillnet@quillnet-gateway:~ $
```

(Source: The team's own work)

Public network

Access network (WAN): the gateway gets Internet through its wireless interface (wlan0), connected to a phone's hotspot that stands in for the internet provider in the demonstration. Because it is behind NAT, the gateway does not receive incoming connections: it initiates all connections to the backend.

Backend on Render: a service with a public address, protected by HTTPS with HSTS, security headers (CSP, X-Frame-Options), CORS with an explicit list of origins, JWT authentication for people, its own token for the Raspberry Pi, and rate limiting; it is the only Quillnet component visible from the Internet. The external services Groq (alert translation) and Resend (email) are consumed only by the backend. The frontend is accessed from any browser through the public URL.

Security analysis

1. Project overview

- Name: Quillnet
- Problem it solves: home IoT devices (cameras, voice assistants, smart TVs) are usually connected to the same flat network as users' computers/phones, with no traffic control between them, becoming an entry point for attacks. Real cases such as the Aisuru-Kimwolf botnet (31.4 Tbps, formed largely of compromised home devices) or BadBox 2.0 (millions of compromised smart TVs) show the real impact of this problem.
- Users: homes with IoT devices.
- Technologies: Raspberry Pi 4 (gateway), Raspberry Pi OS Lite (Debian 13), nftables, dnsmasq, Suricata 7.0.10 (inline IPS), 802.1Q VLANs, Flask + Gunicorn and PostgreSQL on Render (backend), Groq API (alert translation), Resend (email), and an HTML/CSS/JS frontend.
- Type of information it processes: network traffic metadata (IPs, ports, protocols, Suricata alerts), not the content of communications. Also account data (credentials with scrypt hash), devices, and alerts in PostgreSQL.
- Devices/services involved: RPi4 gateway, TP-Link EAP110 AP, real IoT devices per VLAN, Kali Linux VM (pentesting), external Groq API, WAN via cell phone hotspot.

2. Scope of the security analysis

What is evaluated:

- Local network (VLAN segmentation, nftables firewall, Suricata detection)
- Flask/PostgreSQL backend
- AI API (Groq) as an external integration
- Frontend (dashboard)
- TP-Link EAP110 AP

What is not evaluated (and why):

- Security of the cell phone hotspot as a WAN provider — it is third-party infrastructure used as an ISP stand-in for the demo, outside the team's control.
- Internal firmware security of the IoT devices themselves — the project treats them as untrusted by design; it seeks to contain them, not to patch them.
- Groq's infrastructure — only how Quillnet consumes it is evaluated, not its internal security.

3. Asset identification

Asset	Importance	Owner	Consequence if compromised
RPi4 (gateway)	Critical	Segmentation and security of the home network.	Total loss of segmentation and detection — the network becomes flat again
nftables rules / VLAN config	Critical	Devices, isolated into categories and secure.	Loss of isolation between devices
Suricata logs (eve.json)	High	Reports of attacks carried out	No evidence of attacks and no audit capability
PostgreSQL database (alerts and accounts)	High	Alerts of attacks carried out against the network	Loss of history; tampering could hide a real attack
Groq API key	High	Alert translation	Quota abuse or unauthorized use in the project's name

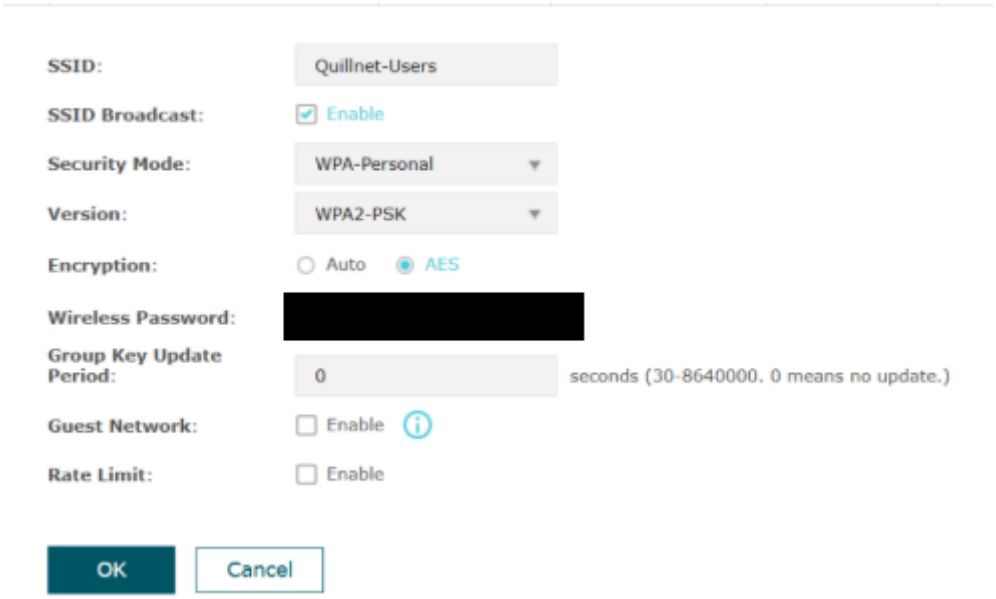
Source code (repo)	Medium-High	Detection logic	Leak of detection logic or hardcoded credentials
TP-Link AP credentials	High	Isolation configuration	Malicious reconfiguration that nullifies isolation
Documentation (Notion, manuals)	Medium	Organized progress	Affects the project's continuity
Project image and reputation	Medium	Project identity	Damage to credibility before the judges
Operational secrets (gateway_secret, JWT_SECRET_KEY, Groq and Resend keys, database password)	Critical	Authentication of the Pi and of people; access to external services	Impersonation of the gateway or of users, quota abuse, or access to the database
Gateway identity file (gateway.json)	Critical	Authenticate the Pi to the cloud	Whoever copies it could send alerts and commands as if they were the Pi

4. CIA triad analysis

Confidentiality

- **What is private?** Suricata alerts (they reveal which devices exist and their behavior), Groq API key, AP and dashboard credentials.
- **Who has access?** The technical team to the gateway (via SSH); the end user to the dashboard.
- **How is it controlled?** SSH access to the RPi4; VLAN segmentation limits who can see whose traffic.
- Are passwords/sensitive data protected? User passwords are stored with an scrypt hash; the Groq API key and the other secrets are read from environment variables and not from the code. Login also requires a 6-digit code sent by email (stored only as HMAC-SHA256). Secrets exposed during development must be rotated.
- Encryption? The backend on Render is served over HTTPS, so connections from the browser and from the gateway to the backend travel encrypted. On the wireless network, the

four SSIDs use WPA2-PSK with AES encryption and a different password per VLAN, so compromising the camera network does not give access to the users' network. As limitations, WPA2-PSK uses a key shared per network and the access point does not rotate the group key (update interval set to 0).



(Source: The team's own work)

- Additional controls implemented: the gateway's secret is stored in the backend only as a SHA-256 hash; verification codes are stored as HMAC; recovery tokens are stored as SHA-256 and travel in the URL fragment (#) so they do not end up in the server logs; and the AI is never sent IPs, MACs, or names.

Compliance: VLAN segmentation prevents a device on VLAN 20 from seeing VLAN 10 traffic, even though they share the same physical network. Problem: without strong authentication on administrative access, a compromise of that credential nullifies the entire architecture. Improvement: SSH key authentication (not password) + API key in an environment variable. Additional problem: the gateway does not yet have an input chain in nftables, so its own services (for example SSH) could be reachable from the IoT segments. Improvement: input chain with a drop policy and SSH by key only from the users' VLAN.

Integrity

- Who modifies data? Only Suricata (on the gateway) writes to eve.json; only the backend writes to PostgreSQL.
- **How is tampering prevented?** The technical documentation shows no evidence of an integrity control (read-only permissions, hashes) on the logs once written.
- **Are changes logged?** Suricata's logs record network events, but there is no evidence of logging of administrative changes to the system itself (e.g., who modified an nftables rule and when). Audit records are kept in the database: the verification challenges table, the recovery tokens, and the command queue (with its manual or automatic origin).
- Data validation? The backend validates and sanitizes input data; in addition, the artificial intelligence receives only the alert's technical text, without IP addresses.
- Backups? The database is a managed service on Render. A full backup was made with pg_dump (October 26, 2026). There is no record of a copy of the gateway configuration outside the original microSD; creating one is a pending improvement in the plan.

Compliance: the detection pipeline (Suricata → eve.json → backend) has a single source of truth. Problem: there is no integrity control over the logs once they are generated. Improvement: read-only permissions on eve.json and periodic backup of the nftables/VLAN configuration. **Availability**

- **What would bring the project down?** Failure of the RPi4 (single hardware unit, no redundancy), loss of the WAN connection, microSD corruption.
- **Backup?** The original microSD with the previous configuration is kept intact as a functional backup. There is no record of a backup image of the current state (VLANs, nftables, Suricata); creating one is a pending improvement in the plan.
- **Recovery?** Without an up-to-date backup image, recovery would mean reconfiguring everything manually — a real risk ahead of a live demo.
- **What happens if the Internet/server/power fails**? If the WAN fails, internal segmentation should keep working — it is worth confirming this with an explicit test (disconnect the WAN and verify that inter-VLAN traffic is still blocked). Test carried out: the gateway's services and rules are restored after a reboot and after a power cut. This is because the three own services have automatic restart (Restart=always) and because, at startup, quillnet-control

asks the backend which devices must remain blocked and reapplies their rules, since nftables does not keep dynamic rules across reboots. In addition, the November 4, 2026 logs show that, upon the cold start of Render's free plan (timeouts expired), the scripts catch the error, log it, and retry in the next cycle without stopping.

- **Redundancy?** There is no hardware redundancy (a single RPi4) — acceptable in an educational prototype, but it must be declared as a known limitation.

In addition, there is a line of work under evaluation on unifying the modem, the gateway, and the AP into a single physical device. This is not a security vulnerability, but a design decision with a direct implication for availability: it would reduce points of failure from external cabling, but would concentrate the entire system in a single point of total failure. It is a trade-off that must be evaluated before it is implemented, not something that has already been resolved.

Compliance: a functional backup copy of a previously known configuration exists. Problem: there is no evidence of an up-to-date backup image of the current configuration state. Improvement: clone the current microSD once the configuration is stabilized, before Expotec (November 13 to 15, 2026).

5. Vulnerability identification

Vulnerability	Affected component	Possible consequence	Ease of exploitation
Suricata in IPS mode with bypass (fail-open)	Prevention layer	If the engine fails, traffic passes through without inspection	Low — design decision that prioritizes availability
VLAN hopping	Network segmentation	If it were possible, it breaks the central pillar of the architecture	Thanks to segmentation, it was not possible
Secrets exposed during development (JWT key, Groq key, database password)	Backend and external services	Impersonation of sessions, quota abuse, or access to the database	Medium: they are rotated at the end of the demonstrations so as not to break them
Lack of integrity control on logs	eve.json / PostgreSQL	Tampering with evidence by someone with access to the system	Medium (requires prior access)
No up-to-date backup	Entire gateway	Slow recovery after a	N/A — operational gap

image		hardware/SD failure	
AP/gateway admin authentication unconfirmed	EAP110 AP, RPi4	Unauthorized access to the VLAN configuration	Depends on whether default passwords were changed
No device identity verification per VLAN	Segmentation	Any device connected to the right SSID inherits the trust of that VLAN	Medium — requires no exploit, only physical proximity
No input chain in nftables	Gateway (RPi4)	The Pi's services reachable from IoT segments	Medium — requires a compromised device on the network
Dependence on free or short-term plans (Render, Groq)	Render / PostgreSQL / Groq	Outages or data loss	Occurred on October 27, 2026; mitigated with a temporary renewal
Detection not verified on VLANs 20, 30, and 40	Suricata	Possible blind spots outside the tested VLAN	Unknown — pending testing
JWT token in the SSE connection URL	Backend / frontend	The token could end up in access logs	Medium
Rate limiter in the memory of a single process	Flask-Limiter	Counters neither shared nor persistent	Low
No explicit JWT revocation	Sessions	A stolen session works for up to 7 days or until the password is changed	Medium
Email sender without its own domain	Resend	Emails (2FA, recovery) reach only the owner of the service account	Medium
Intra-VLAN traffic not inspected	Segmentation / Suricata	A compromised machine can attack another on its own VLAN without the IPS seeing it	Medium: requires no exploit, only being on the same VLAN; mitigation: client isolation on the AP
WPA2-PSK with a key shared per VLAN and no group key rotation	EAP110 access point (2022 firmware)	Whoever knows a VLAN's key gets into that VLAN	Medium: requires knowing the key; the key is different on each VLAN
Encrypted traffic (TLS) not inspected	Suricata	Content signatures do not apply to HTTPS downloads	Medium: known limitation of an IPS without TLS inspection

6. Threat identification

Threat	Vulnerability it exploits	Affected asset	Consequence
DDoS attack from compromised IoT (Aisuru type)	No mitigation of volumetric attacks (the IPS acts by signatures)	Bandwidth, availability	Network saturation, possible disconnection
VLAN hopping	Segmentation not empirically validated	All assets on the 4 VLANs	Total loss of isolation between segments
Theft/leak of the Groq API key	Insecure handling of credentials	The project's Groq account	Quota abuse, unauthorized spending
Unauthorized access to the AP	Weak/default password	VLAN configuration	Malicious reconfiguration that nullifies segmentation
Injection into the LLM (prompt injection via alerts)	Lack of data validation before sending to Groq	Quality of the alerts shown to the user	False or manipulated alerts. Partially mitigated: attack type classification uses fixed rules and a closed list, and the AI receives only the signature, the category, the action, and the protocol.
Hardware failure without an up-to-date backup	No recent backup image	Availability of the entire system	Loss of hours of work, risk of failure at the demo

7. Risk assessment

Risk	Likelihood	Impact	Level	Rationale
Short circuit with the stand's wiring, which could endanger the integrity of the building.	Low	Medium	Medium	The likelihood of something like this situation occurring is low because the building has a wiring system designed to withstand large amounts of power. It is also worth considering the possibility that consumption will be higher because of the various projects in use. Its impact would be low since the thermomagnetic breaker will trip instantly. For that reason its risk level is considered medium.

Problems with the stand's structure at the time of the exhibition, causing it to collapse or fail to support the weight of the decoration.	Low	high	medium	The likelihood of this occurring is low since the structure is planned to support the base cover, such as the tarps or fabric. Although there may be a risk that, when a heavy object such as a television is placed on it, it could collapse. The impact would be high since it would mean the base structure collapsing, leaving no space to present during the fair. For that reason its risk level is medium since, even though its likelihood is low, its impact on the safety of the students and on the project would be high.
Failure of the Raspberry Pi systems with the logs and the PostgreSQL database	Medium	Medium	Medium	The likelihood of a problem occurring with the network services provided by the Raspberry Pi is medium since it could be due to either a misconfiguration or human error. Its impact will be medium since the error would only need to be evaluated and corrected quickly through a short analysis by the technical team.
Hardware failure without a backup prior to the accident.	Low	High	Medium	The likelihood of an accident that leaves the main device without a system and without a prior backup is very low, first because the device would be protected by a protective case, and also because its basic configuration for normal operation would take only a short time to set up. Its impact at the moment would be high because it could affect the base or already-configured systems at the time the device is in use, leaving the networks unprotected.
Use of VLANs or protection and segmentation systems that are not approved or verified.	Medium	Medium	Medium	On the Quillnet device, the likelihood that the VLANs are not approved or do not have a verified system is medium since continuous pentesting is carried out to verify their security. It is true that the possibility of these VLANs being breached by an attacker must also be considered. Its impact would be large since it would be a way to exploit a system vulnerability that

				can be critical for users. For that reason its impact level will be Medium.
Expiration of Render's free database (October 27, 2026)	Medium	High	High	The risk materialized on October 27, 2026 and was mitigated with a time-limited paid renewal; if that renewal expires before the presentation, the backend loses its database.
Gateway reachable from IoT devices due to the absence of an input chain	Medium	High	High	A compromised device could try to access services on the Raspberry Pi.

8. Security measures

Risk addressed	Proposed measure	Type of measure	Owner	Priority
Short circuit with the stand's wiring, which could endanger the integrity of the building.	Replacement equipment	Recovery measure	The project's risk analyst	High
Problems with the stand's structure at the time of the exhibition, causing it to collapse or fail to support the weight of the decoration.	Recovery plan	Recovery measure	The project's risk analyst	High
Failure of the Raspberry Pi systems with the logs and the PostgreSQL database	Data validation	Preventive measure	Those in charge of the technical development and programming side.	High
Hardware failure without a backup prior to the accident.	Backups	Recovery measure	Those in charge of the project's network infrastructure.	High
Use of VLANs or protection and segmentation	Alerts.	Detection measures	Those in charge of the programming and	High

systems that are not approved or verified.			network infrastructure side of the project.	
Database expiration / exposed secrets	Migrate or renew PostgreSQL and rotate the secrets used during development. Status: database renewed; secret rotation scheduled after the last demonstration.	Preventive and recovery measure	Web development leads	High
Gateway reachable from IoT devices	Input chain with a drop policy and SSH by key only from the users' VLAN	Preventive measure	Network infrastructure leads	High

9. Access security and user control

- **What types of users exist?** There are two authenticated actors and one projected profile. (1) The household member: has an account with email and password, linked to a gateway through a single-use 8-character pairing code that expires in 15 minutes; they can view the map, the dashboard, and the alerts, approve, block, and unlock devices, and change their settings, and they see only their own household's data. (2) The Raspberry Pi: authenticates with its own secret (not with a username and password) and can only send alerts and presence, check its command queue, and acknowledge commands. (3) The ISP technician (projected profile): would have a multi-household support console; it is not implemented in the prototype and remains future work. Guests who connect to the Wi-Fi have no account or access to the dashboard; their traffic is subject to the rules of the VLAN they belong to.
- **How do people register and log in?** The person creates their account with an email and password; the email is validated (syntax, the domain's MX/A/AAAA records, a list of disposable domains, and suggestions for correcting common mistakes). To link their

household they enter the pairing code. Login issues a 7-day JWT that includes the gateway identifier, and with incorrect credentials a generic message is shown.

- **How are passwords recovered?** With “Forgot my password,” a single-use link that expires in 30 minutes is sent to the email address. The system responds the same whether or not the account exists, so as not to reveal which emails are registered; the token is stored only as a SHA-256 hash and travels in the URL fragment. The password can only be changed through that link and, when it is, previous sessions are invalidated.
- **How is a device or account blocked?** Any household member with an active session can block a device immediately (the Raspberry Pi checks for commands every 5 seconds). In addition, the system applies automatic quarantine when a single device generates 3 real-block alerts in 5 minutes. Unlocking requires two-step verification: password and a 6-digit code sent by email (10 minutes, maximum 5 attempts). The asymmetry is intentional: blocking reduces risk; unlocking reopens it.
- **What happens when a user stops using the system?** The session (JWT) expires after 7 days and a password change invalidates sessions issued before it. There is not yet explicit revocation of individual tokens (a known limitation, see the improvement plan).
- **How is the principle of least privilege applied?** On three levels: each account accesses only the data of its own gateway (every query filters by gateway identifier and a foreign one returns 404); the Raspberry Pi can only use the gateway endpoints and its secret is stored only as a hash; and on the gateway itself quillnet-agente and quillnet-vigia run under an unprivileged user, and only quillnet-control runs as root because it needs to manipulate nftables.

Function	Household member	Raspberry Pi (gateway)	ISP technician (projected, not implemented)
View the map,	Allowed	Not applicable	Projected

dashboard, and alerts of their household			(multi-household)
Approve a new device	Allowed	No	Projected
Block a device	Allowed (direct)	Executes the command	Projected
Unlock a device	Allowed with two-step verification	Executes the command	Projected
Send alerts and presence	No	Allowed	No
Change the password	Only through a single-use link by email	No	No
Delete alerts	Allowed (soft delete; they still count in the statistics)	No	Projected
Modify VLANs or firewall rules	No (there is no screen for it)	Configured over SSH	Projected

10. Data protection and privacy

- **What personal data is collected?** Since Quillnet is a system that intercepts network traffic, it collects only network traffic information; for security reasons it does not store the data or requests of the network. Only the delivery data and the IPs that are necessary. It also stores the configuration of the VLANs and the connected devices. Account data (email, name, hashed password, province and canton of the household) and device data (MAC address, IP, name, and last seen).
- **What is it used for?** All the information is stored as a traffic log in order to be able to intercept suspicious requests. Devices are also stored as a log.
-
- **Where is it stored?** In a PostgreSQL database hosted on Render (a cloud provider), shared among households and logically separated by gateway identifier (multi-tenant). In the projected model, this database could operate under the ISP's responsibility. Data travels encrypted over HTTPS.

- **Who can access it?** Each person accesses only the data of their own household. The development team has access to the database through operational credentials.
- **How long will it be kept?** Alerts are kept for 90 days; an automatic retention process deletes them when that period expires, along with tokens and verification codes that expired more than 30 days ago. Both periods are configurable.
- **How will it be securely deleted?** The person can delete alerts from their inbox: it is a soft delete (the alert stops being displayed but is kept for the statistics until the retention period ends). When the 90 days expire, the retention process deletes them from the database. Backups (pg_dump) keep copies until they are manually deleted
- **What data should not be collected?** Data regarding users' accounts on other systems, tokens, cookies, passwords, requests to servers, specific actions on a website, and other data that must not be collected should not be collected.

11. Secure development

- **How is input data validated?** The backend validates inputs and accesses the database through SQLAlchemy with parameterized queries (the project's development guidelines require parameters and forbid building SQL queries with f-strings), which prevents SQL injection; the Content-Security-Policy header restricts the allowed scripts, mitigating XSS.
- **How are errors handled?** They are handled carefully so that the system does not collapse if an error occurs; functional tests of the project are run frequently to verify that the systems operate without complications.
- **How are credentials protected?** Through secure storage in the databases and a hashing system in which passwords are stored securely using strict cryptographic algorithms.
- **Are there passwords written directly in the code?** No. Secrets are read from environment variables (on Render) or from local .env files excluded from the repository; on the gateway, the secret is in gateway.json, outside the repository. The secrets that were exposed during development will be rotated at the end of the demonstrations.

- **How are dependencies updated?** Through package managers such as pip in Python, which is the language used for the backend. Keeping dependencies up to date and verifying their compatibility so that there are no cross-errors between two or more dependencies.
- **How is access to the source code controlled?** Absolutely no home user has access to the source code. Technical users have access only to the basics of the software. Only the main developers have access to the source code in order to modify it, under strict control in which every change to the system's infrastructure must be documented.
- **What security tests were performed?** Manual detection and blocking tests were performed from Kali Linux (a real Covenant beacon, unencrypted MQTT, sqlmap, and others; see section 14) along with 238 automated backend tests that include security cases (for example, it was verified that the test fails if the verification code is stored with a simple hash). The formal per-VLAN pentesting battery, the VLAN hopping test, and the load tests are still pending.

12. Backups

- **What information needs a backup?** Passwords, users, emails, and networks linked to a Quillnet device. These data are stored in the database and need a backup since they are what is needed to authenticate users and protect the networks that have a Quillnet device. In addition to the database, the gateway configuration (VLANs, nftables, dnsmasq, Suricata, and systemd units) and its identity file are backed up.
- **How often will the copy be made?** Prototype: manual backup with pg_dump; the last full one was made on October 26, 2026 . Product (projected): automatic weekly copy. So that the stored data is safely kept in a copy, preventing failures in the devices that store it.
- **Where will it be stored?** It will be stored in the cloud to guarantee its security. This means that no data needs to be stored on physical devices belonging to the ISPs. This allows more accessible control over the device.
- **Who will be responsible?** The digital responsibility for keeping the data safe will fall to the ISPs' technicians or cybersecurity specialists. Meanwhile, those responsible for the physical security of the devices that store this information are those in charge of the cloud servers.

- **How will it be verified that the copy works?** By testing the contents of these copies to verify that all the data is being stored intact and without any kind of change. These tests are performed right after storing the copy to verify its state.
- **How long would it take to recover the project?** Backend: a new deployment on Render in about 2 minutes and database restoration from pg_dump in 5 to 10 minutes . Gateway: with the backup microSD, 10 to 15 minutes; without an up-to-date image it would be necessary to reconfigure manually (several hours).

13. Incident response

- **Step 1:** The incident must be identified; to do this, specialists are consulted to trace the root of the problem. Afterwards, the information security staff or the SOC is informed so that recovery and documentation protocols are started, in addition to notifying the corresponding ISP authority
- **Step 2:** Once the error has been identified, it must be contained through protocols that are either physical (such as protecting devices with security guards, among others) or computer-based methods (closing ports, blocking access, among others). At the same time, another team investigates more deeply to monitor the SOC's actions and make sure it is not a False positive or a False negative. Immediate backups of the information must be made, in addition to preserving the evidence related to the incident. Once the cause has been identified, it must be eliminated and further occurrences prevented.
- **Step 3:** Once the cause has been eliminated, all services must be tested to ensure their integrity; if one of them has suffered damage, it will need to be recovered as soon as possible, whether through backups or from the code. The operations team must document a report specifying each of the protocols carried out to recover the system. The people in charge must also inform the respective departments to prevent these kinds of accidents from happening again.
- **Application in Quillnet:** (1) Identify: the agent sends the alert and the backend classifies it into one of 13 types. (2) Contain: automatic quarantine blocks the device by MAC address after 3 real-block alerts in 5 minutes, or the person blocks it manually. (3) Recover: unlocking

requires two-step verification; upon suspicion of a compromised account, the password is changed through a single-use link, which invalidates previous sessions; upon the leak of a secret, the JWT key, the Groq and Resend keys, and the database password are rotated.

14. Results analysis

Threat detection and blocking tests

The following tests make it possible to verify Quillnet's ability to detect and block suspicious communications generated by a compromised IoT device. Suricata in IPS mode was used to intercept traffic and record security events.

The tests of the most recent battery were run on November 5, 2026 from a Kali machine (192.168.10.171) located on VLAN 10, with the tmNIDS framework (testmynids.org) and tools such as curl, sqlmap, and mosquitto_pub. The evidence was obtained by filtering eve.json with jq and observing the dashboard and the app. A distinction is made between confirmed results (with a log line) and results pending evidence.

Test	**Objective**	**Result**	**Evidence**	**Improvement applied**
Attack 1: Automated SQL injection (sqlmap)	Simulate the reconnaissance and automated exploitation that any web panel exposed on the home network would receive (e.g., the administration interface of a camera or IoT device).	The request completed (sqlmap received a response from the test server). The November 5 report shows that, against a machine on the same VLAN 10, no alert was generated: traffic within the same VLAN does not pass through the gateway and Suricata does not	sqlmap -u "http://192.168.10.157/?id=1" --batch --level=1	Demonstrates the network's ability to detect automated exploitation tools in real time.

		inspect it (a finding included in the improvement plan). Detection of signature 9000001 requires the traffic to cross the gateway; that variant remains pending to be repeated with evidence.		
Attack 2: Simulated malware check-in	Simulate the moment when a compromised device reports its data to a remote server to confirm that it is still "alive" and receive instructions (a classic pattern of malware based on NSIS installers).	The connection did not complete; both signatures fired and the event log confirms the blocking action for both. The November 5 report records this test as pending photographic evidence.	curl -X POST "http://1.1.1.1/temp/gate.php" -A "NSIS_Inetc" -d "rms_id=1&right=SYSTEM&os=test&hostname=test&domain=test" --max-time 3	Demonstrates the ability for active blocking in a post-exploitation scenario (a device trying to communicate outward).
Test 2: real C2 beacon (Covenant)	Verify the inline blocking of a real beacon from the Covenant framework (ET MALWARE Covenant Framework HTTP Beacon, sid 2027793), generated from the Kali machine on VLAN 10.	Blocked inline (action: blocked), severity 1 (high); the dashboard classified it as Malware and the app notified it. Confirmed.	/tmp/tmNIDS -15 (tmNIDS, testmynids.org).	None required; the rule is in drop mode (12,860 rules in drop).
Test 5: unencrypted MQTT (port 1883)	Observe what happens with a publish to a public MQTT broker without TLS, a risky	The command hung, port 1883 was unreachable, and there was no alert; the traffic did not	mosquitto_pub -h test.mosquitto.org -t casa/quillnet-test -m "reboot";	Repeat from another network to attribute

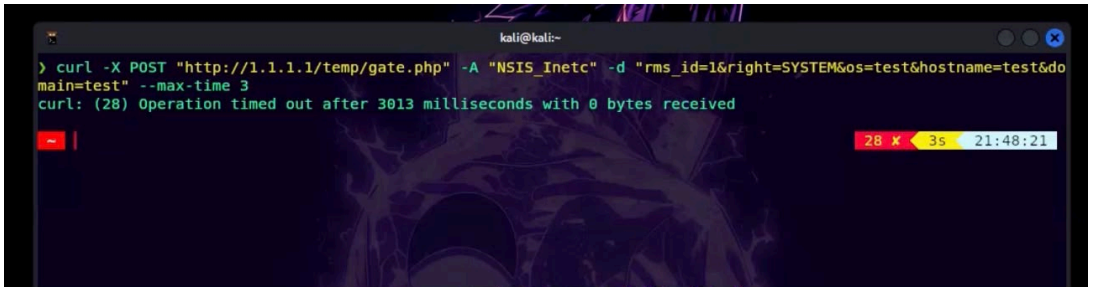
	practice typical in IoT.	leave the network. Confirmed in the November 5 report; it was not determined whether the cut was made by the gateway or by the mobile carrier’s network.	timeout 5 bash -c 'echo > /dev/tcp/test.mosquitto.org/1883'.	the cut.
Test 6: sqlmap against a machine on the same VLAN	Verify whether Quillnet detects a web attack launched within the VLAN (custom rule 9000001).	No alert against 192.168.10.123: traffic between machines on the same VLAN does not pass through the gateway. Confirmed (external variant pending).	sqlmap against the local machine on VLAN 10.	Client isolation on the access point.
Test 8: dropper (executable download)	Detect the download of a test executable (tmNIDS -6).	No alert on the dashboard. Confirmed. Possible causes according to the report: the corresponding signature is informational (severity 3) and Quillnet discards it by design, or the download traveled encrypted (HTTPS) and an IPS without TLS inspection cannot apply content signatures.	tmNIDS -6.	To be determined.
Test 10: “id root” response	Verify the GPL ATTACK_RESPONSE signature (sid 2100498, tmNIDS -1).	It did not fire (the GPL rules are probably absent). Confirmed. The report’s hypothesis: the GPL rule subset is not included in the loaded ET Open rule set.	tmNIDS -1.	Verify whether the GPL rule set should be loaded.
Test 3:	Verify the blocking	Pending evidence (to	curl -m 5	—

remote control (custom rule 9000010)	of a remote-control beacon to an external server.	be confirmed).	"http://1.1.1.1/quillnet-c2".	
Test 4: suspicious traffic (custom rule 9000011)	Verify the blocking of anomalous traffic with no known family pattern.	Pending evidence (to be confirmed).	curl -m 5 "http://1.1.1.1/quillnet-raro".	—
Test 7: risky browsing (Tor)	Detect Tor network traffic (tmNIDS -5).	Pending evidence (to be confirmed).	tmNIDS -5.	—
Test 9: adware/PUP	Detect adware (tmNIDS -14).	Pending evidence (to be confirmed).	tmNIDS -14.	—
Lateral movement: Users to Cameras	Verify that a host on VLAN 10 cannot reach one on VLAN 20; use a real host on VLAN 20 (192.168.20.x), not 192.168.20.1, which is the Raspberry Pi itself.	Pending evidence (to be confirmed).	ping and curl to the VLAN 20 host; sudo nft list ruleset.	—
Automatic quarantine	Verify that 3 real-block alerts in 5 minutes leave the device isolated by its MAC address.	Pending evidence (to be confirmed).	Three consecutive blocked lines and the Kali machine in blocked status on the map.	—

From these tests it is concluded that Quillnet blocks malicious outbound traffic from the Users VLAN inline when a rule in drop mode exists (12,860 of the 52,246 loaded rules), that the insecure MQTT channel to the outside was not available, and that traffic within a single VLAN and encrypted traffic fall outside the current inspection. Tests marked “to be confirmed” are not considered achieved until their evidence is available.

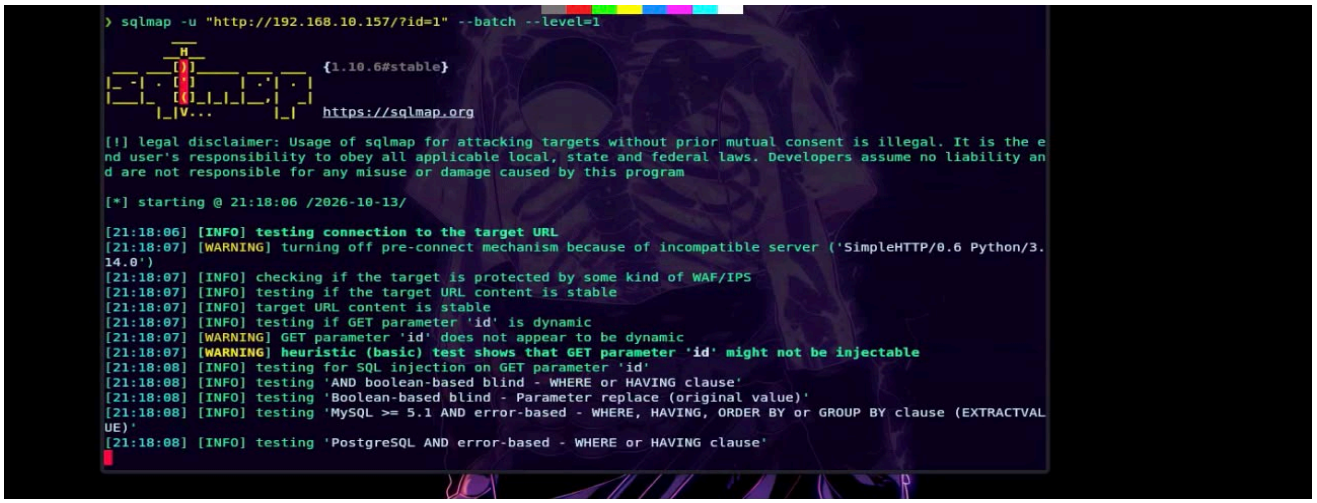
Evidence images

Simulated malware check-in:



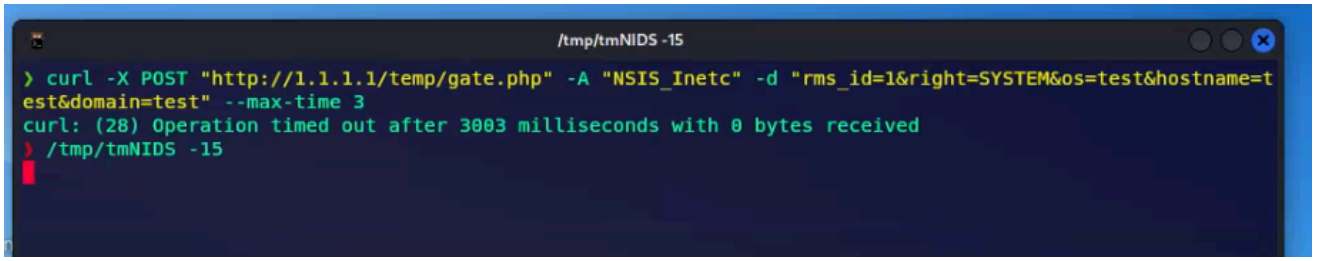
(Source: The team's own work)

Automated SQL injection (sqlmap):



(Source: The team's own work)

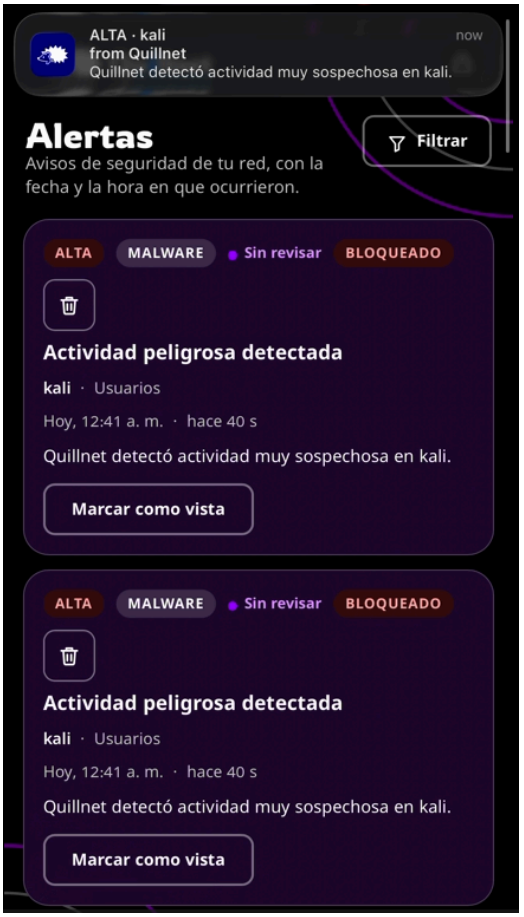
Real C2 beacon (Covenant):



(Source: The team's own work)

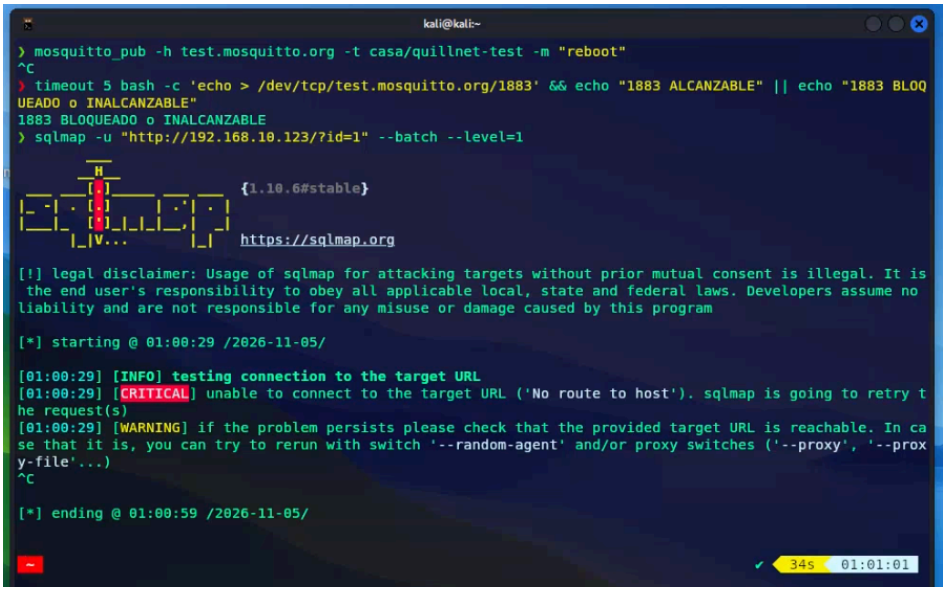
```
quillnet@quillnet-gateway:~ $ sudo tail -f /var/log/suricata/eve.json | jq -c 'select(.event_type=="alert") | {hora:.timestamp, firma:.alert.signature, sid:.alert.signature_id, sev:.alert.severity, accion:.alert.action, src:.src_ip, dst:.dest_ip}'
{"hora":"2026-11-05T06:41:38.402636+0000","firma":"ET MALWARE Trojan Generic - POST To gate.php with no referer","sid":2017930,"sev":1,"accion":"blocked","src":"192.168.10.171","dst":"1.1.1.1"}
{"hora":"2026-11-05T06:41:38.402636+0000","firma":"ET MALWARE Generic Request to gate.php Dotted-Quad","sid":2022986,"sev":1,"accion":"blocked","src":"192.168.10.171","dst":"1.1.1.1"}
{"hora":"2026-11-05T06:41:38.402636+0000","firma":"ET ADWARE_PUP Win32/DealPly.EJ Checkin","sid":2044765,"sev":1,"accion":"blocked","src":"192.168.10.171","dst":"1.1.1.1"}
{"hora":"2026-11-05T06:43:51.750331+0000","firma":"ET MALWARE Covenant Framework HTTP Beacon","sid":2027793,"sev":1,"accion":"blocked","src":"192.168.10.171","dst":"103.224.182.211"}
```

(Source: The team's own work)



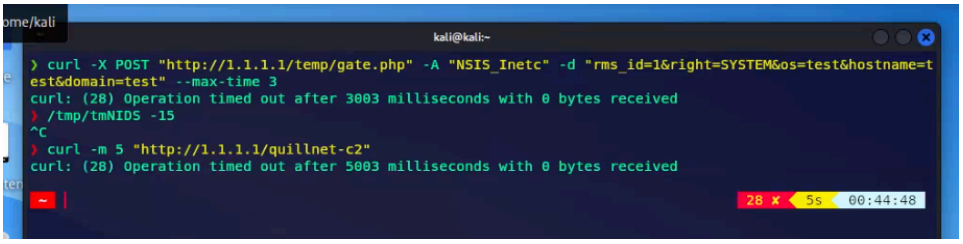
(Source: The team's own work)

sqlmap against a machine on the same VLAN (no alert):

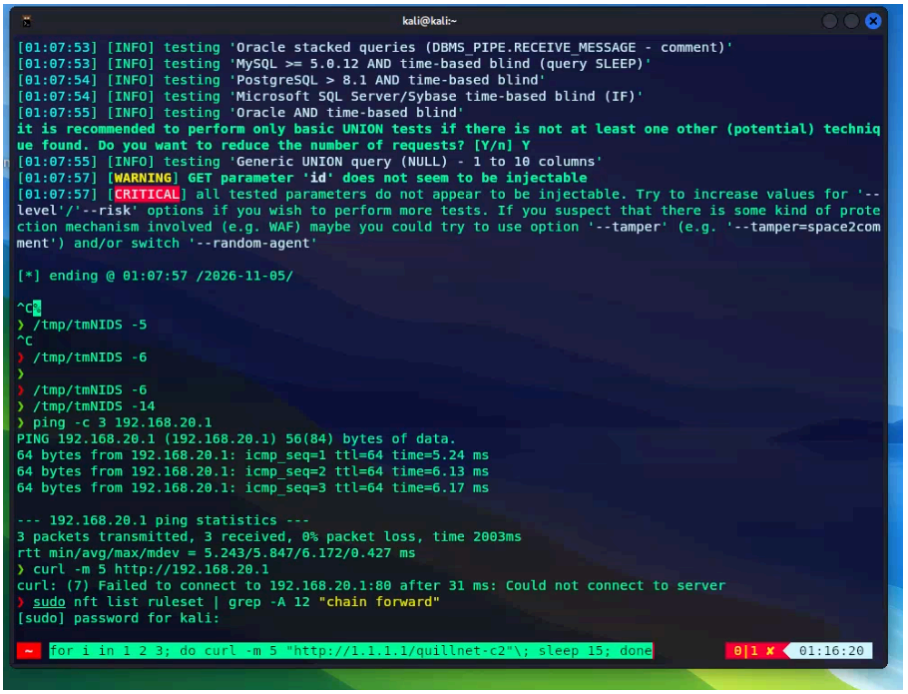


(Source: The team's own work)

Lateral movement Users → Cameras (pending):



(Source: The team's own work)



(Source: The team's own work)

Automatic quarantine (pending):



(Source: The team's own work)



(Source: The team's own work)

15. Improvement plan

Improvement	Priority	Timeframe	Owner
Input chain in nftables with a drop policy and SSH by key only from VLAN 10 (do it with physical access to the Pi in case you get locked out)	HIGH	Immediate	Infrastructure
Clone the microSD card and back up the gateway configuration	HIGH	Immediate	Infrastructure
Test push notifications on a real phone (Android and iPhone with the PWA) and the email flow	HIGH	Immediate	Web development
Rotate the exposed secrets (JWT, Groq, database password) after the last demonstration	HIGH	Immediate	Web development
Verify an own domain in Resend (or confirm that the sender email address belongs to the Resend account owner) so that the login 2FA code reaches any user; in the meantime, LOGIN_2FA=0 as an emergency switch	HIGH	Immediate	Authentication
Migrate the database to a plan with no expiration, or export/restore it	HIGH	Short term	Web development
Formal pentesting per VLAN, VLAN hopping test, and detection on VLANs 20, 30, and 40	HIGH	Short term	Pentesting
Client isolation on the access point (or per-device microsegmentation) in order to see intra-VLAN traffic	HIGH	Short term	Infrastructure
Repeat the MQTT test from another network or link to determine whether the cut was made by the gateway or by the mobile carrier	MEDIUM	Immediate	Pentesting
Load tests and WAN outage tests	MEDIUM	Short term	Pentesting / Infrastructure
Own verified domain in the email service	MEDIUM	Short term	Web development
Read-only eve.json and a history of administrative changes	MEDIUM	Short term	Infrastructure
Review WPA2-PSK: group key rotation and firmware update of the EAP110 (2022)	MEDIUM	Short term	Infrastructure
Rate limiter in shared storage (Redis)	MEDIUM	Long term	Web development
SSE with a single-use ticket instead of the JWT in the URL	MEDIUM	Long term	Web development
Explicit JWT revocation	MEDIUM	Long term	Web development
Rules based on TLS metadata (JA3/SNI) and DNS	MEDIUM	Long term	Infrastructure

filtering for encrypted traffic			
Record the discarded low-severity signatures in the internal project log; verify whether the GPL rule set should be loaded	LOW	Short term	Pentesting
Alert filter by type and date; ISP technician console	LOW	Long term	Web development

16. Conclusions

- **What is the project's greatest risk?** Today the greatest risks are: the pending validation of isolation between VLANs (VLAN hopping) and of detection on VLANs 20, 30, and 40; the lack of an input chain on the gateway; and the dependence on free-tier or short-term services (the Render database expired on October 27, 2026 and was renewed for a limited time). The absence of an up-to-date microSD image worsens the impact of a hardware failure.
- **What asset needs the most protection?** The most important asset is the Raspberry Pi, which controls all the network traffic in the households.
- **Which principle of the CIA triad is best applied?** Confidentiality at the application layer: the Pi's secrets stored only as hashes, hashed passwords that can be changed only through an email link, two-step verification to unlock, isolation per household, and an AI that never receives IPs or MAC addresses.
- **What needs to be improved?** Availability and integrity: the Raspberry Pi is a single point of failure, there is no up-to-date microSD image, eve.json has no integrity control, and the service depends on free plans; also the confidentiality of the gateway itself until the input chain is added.
- **Which measures should be implemented first?** The input chain with a drop policy and SSH by key, the microSD image and the configuration backup, the rotation of secrets once the demonstrations are over, and the formal pentesting per VLAN.

- **Is the project considered safe to use? Why?:** It is safe as a proof of concept in a controlled environment: the inline blocking of a real Covenant beacon was verified, as were

the fact that unencrypted MQTT traffic did not leave the network, blocking by MAC address, and persistence after a reboot. Its known limits: it does not inspect traffic between hosts on the same VLAN or TLS-encrypted traffic. It should not be considered ready for a commercial deployment until the validation of isolation between VLANs, the load tests, and the closing of the gaps listed have been completed.

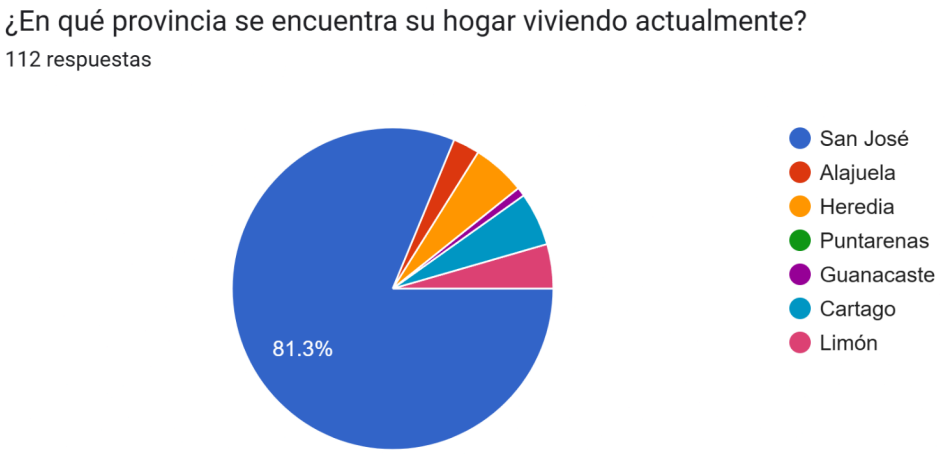
Chapter IV

Results analysis

The analysis of the project's results will be divided into two important sections: the first will focus on analyzing the results of the surveys carried out in order to interpret the need for a device like Quillnet in domestic households. The second part will analyze the security tests of the device.

The survey collected information from 112 different households, which provides broad and varied information about security and devices in the homes of Costa Rican society. The following analysis is based on the answers provided in the form; the sample consisted of 112 households of Costa Rican people from the middle and upper classes.

The first question shows the province of origin of the participants. The results showed that 81.3% of respondents currently live in the province of San José.



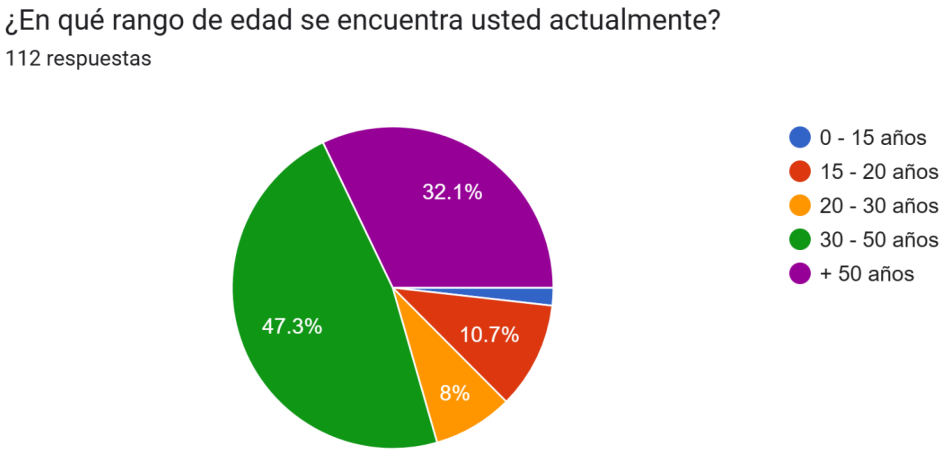
Province of residence of the respondents. Taken from the Quillnet Project 2026 form

The second question asked how many people live in the household. From these data it can be concluded that 61.1% of the sample lives in a household with between 1 and 3 people.



Number of people living in the household. Taken from the Quillnet Project 2026 form

The fourth question asked about the respondents' age range. Based on the data shown in the survey, it can be concluded that 47.3% of household members are between 30 and 50 years old.

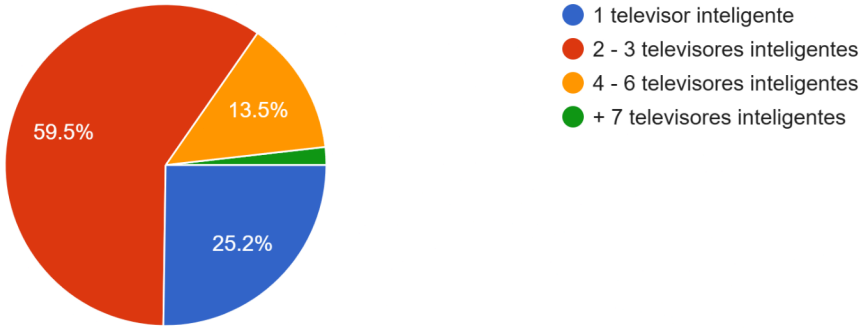


Age range of the people surveyed. Taken from the Quillnet Project 2026 form

The following questions in the form seek to find out which IoT devices are the most common in Costa Rican households among the middle and upper classes. From the participants' answers it can be concluded that the most common devices in home environments are smart TVs, cell phones, computers, among others. This reflects that internet-connected systems are becoming more common among the population of Costa Rica.

En caso de tener televisores inteligentes en su hogar. ¿Cuántos tiene?

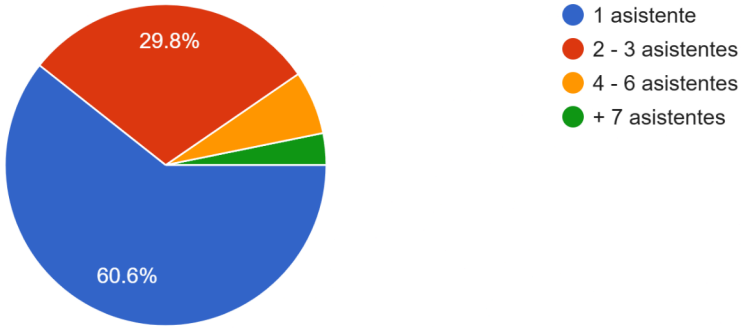
111 respuestas



Number of smart TVs in the household. Taken from the Quillnet Project 2026 form

En caso de tener asistentes de voz (como parlantes inteligentes) en su hogar . ¿Cuántos tiene?

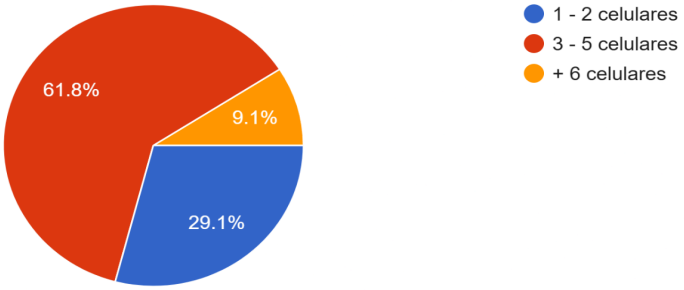
94 respuestas



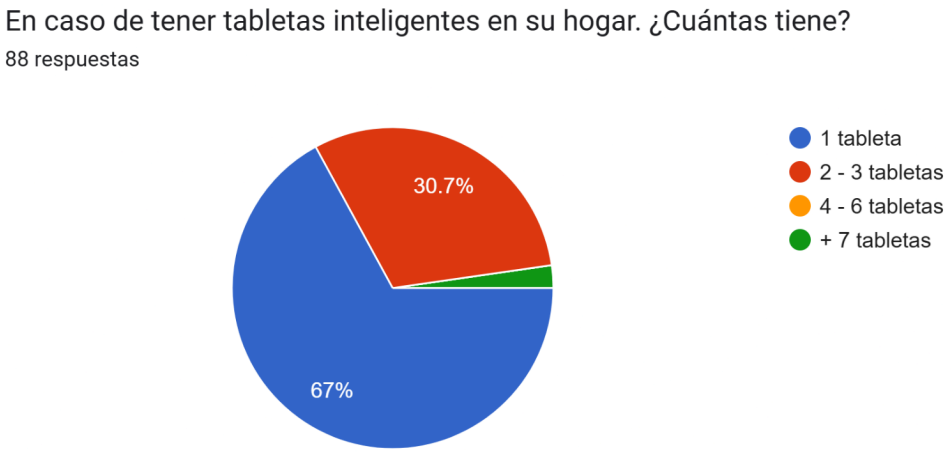
Number of voice assistants in the household. Taken from the Quillnet Project 2026 form

En caso de tener celulares en su hogar. ¿Cuántos tiene?

110 respuestas



Number of cell phones in the household. Taken from the Quillnet Project 2026 form

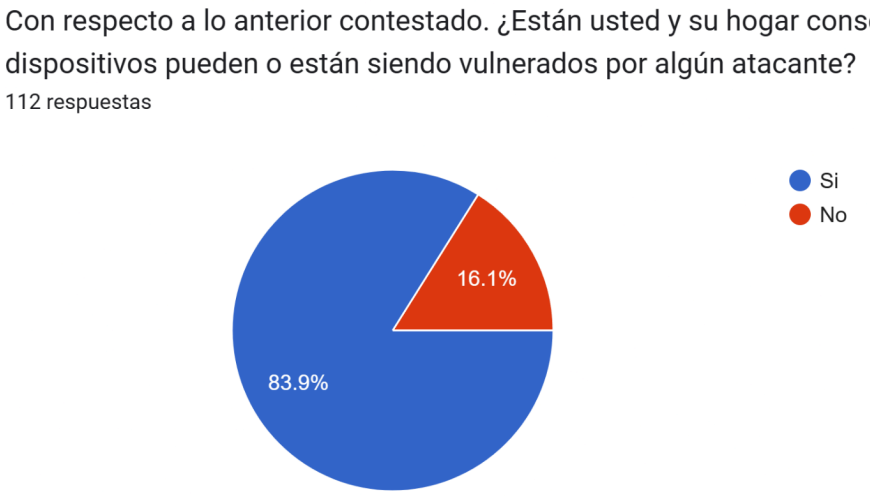


Number of smart tablets in the household. Taken from the Quillnet Project 2026 form

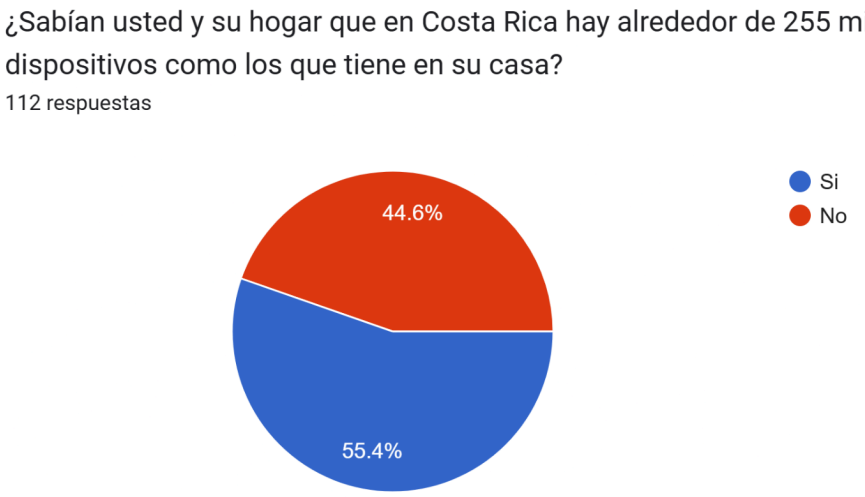


Number of computers in the household. Taken from the Quillnet Project 2026 form

The next two questions, answered by the sample, sought to find out how aware the participants were of their technological vulnerability to cyberattacks in their homes.



Awareness of the vulnerability of technological devices. Taken from the Quillnet Project 2026 form

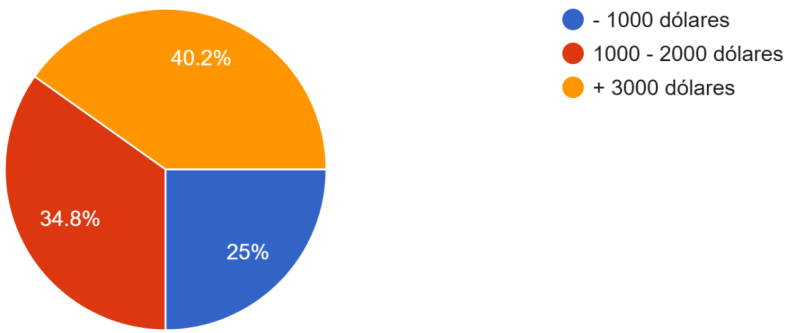


Awareness of the vulnerability of technological devices. Taken from the Quillnet Project 2026 form

For the next question it is important to consider that the data provided in this survey are completely confidential. Participants were asked about their monthly income in order to determine their social class or income level and to assess the possibility of installing Quillnet in both low-income and high-income households.

¿Cuántos son los ingresos mensuales de su hogar? Recuerde que toda la información compartida aquí es totalmente confidencial y no se registraran datos personales de cada hogar.

112 respuestas

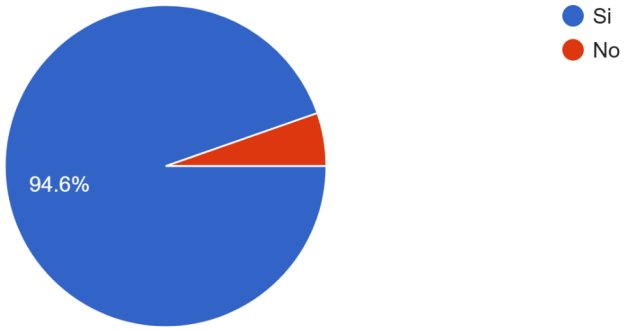


Monthly income of the respondents. Taken from the Quillnet Project 2026 form

The third-to-last question was designed to evaluate the Quillnet project as a feasible and profitable business. In it, participants were asked whether they have used a service that provides them with internet and also allows them to obtain data about their network and protect it from attacks.

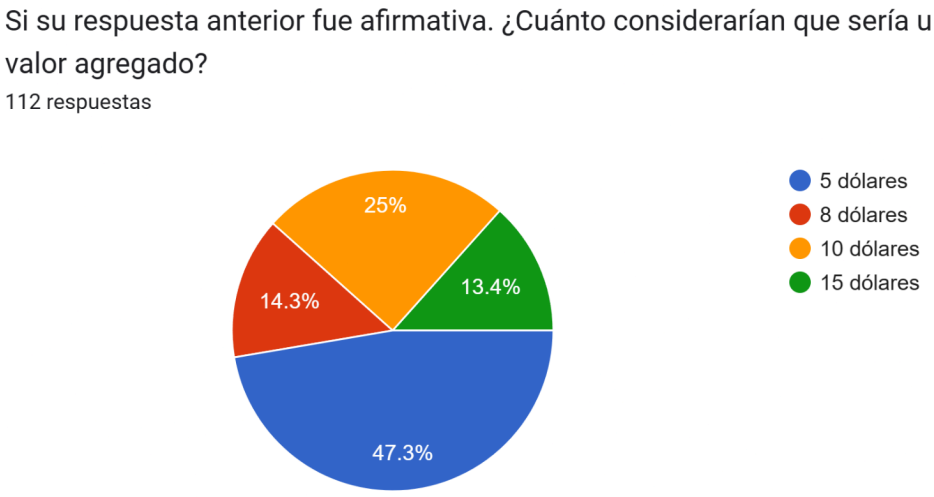
Si existiera un servicio que pudiera proteger sus redes de estos ataques y darle información relevante del estado de sus conexiones integrado a...ternet por un pequeño valor agregado, ¿lo usaría?

112 respuestas



Validation of Quillnet as a business idea. Taken from the Quillnet Project 2026 form

In the last question, participants were asked what a good monthly amount would be to protect their home networks with a Quillnet device. From this it can be concluded that an acceptable price for users would be a 5-dollar increase in the service.



Price for Quillnet as a service. Taken from the Quillnet Project 2026 form

From all this information it could be concluded and interpreted that the Costa Rican population between 20 and 50 years of age, who live mostly in the province of San José, are aware of the cyber dangers that their internet-connected devices may face. It can also be concluded that most users would be willing to pay between 5 and 10 dollars more to protect their internet from threats.

This range includes the projected price of ₡3,500 per month (Chapter III, Administrative and Financial Management), so the model would not require an ISP subsidy to be accessible to the surveyed households. Sample limitation: the 112 responses come from a non-probabilistic sample, concentrated in San José (81.3%) and in middle- and upper-class households, so the results describe that population and cannot be generalized to all households in Costa Rica.

General Conclusions

1. **The security of residential IoT devices represents an important need.** The research carried out made it possible to identify that internet-connected devices can present vulnerabilities that compromise users' privacy. For this reason, QUILLNET proposes an alternative aimed at strengthening the protection of households through network security mechanisms.
2. **The combination of Zero Trust and network segmentation is a solid foundation for protecting devices.** The proposed architecture makes it possible to separate IoT devices from the network used by users and to control communications between segments. This helps reduce the chances of an attacker's lateral movement and limits the impact that the compromise of a device could have.
3. **The integration of detection, analysis, and response tools is one of QUILLNET's main contributions.** The project contemplates the use of Suricata to analyze network traffic, a backend architecture to process events, and an interface that presents alerts in an understandable way. In addition, the Purple Team methodology provides an approach to evaluate defenses through simulated attacks and to improve protection measures according to the results obtained.
4. **Ease of use is a fundamental element for bringing cybersecurity closer to households.** The proposal of a self-management interface, with visual information, alerts, and explanations in plain language, seeks to let people understand the state of their network without needing advanced technical knowledge. In this way, QUILLNET seeks to bring security mechanisms used in business environments closer to a domestic context.
5. **The development of the prototype made it possible to complete advances and to recognize challenges.** The advances include the physical gateway with Raspberry Pi OS Lite, the four VLANs with a Zero Trust firewall, Suricata 7.0.10 in inline IPS mode, the verified

inline blocking of a real beacon from the Covenant framework (with 12,860 Emerging Threats Open rules in drop mode), blocking by MAC address with automatic quarantine and persistence after a reboot, the cloud backend with AI-based alert translation and classification into 13 attack types, the self-management frontend (map, dashboard, and real-time alerts), and 238 automated tests. As challenges, the formal pentesting battery per VLAN, the VLAN hopping test, detection on VLANs 20, 30, and 40, and the performance test under load remain to be completed.

6. **QUILLNET shows growth potential, although it still requires validation before a large-scale implementation.** Its design allows future improvements in automation, monitoring, management of multiple installations, and threat response to be proposed. However, to evolve from a proof of concept into a commercial service it will be necessary to verify its effectiveness through controlled tests, strengthen data confidentiality, evaluate performance, and guarantee the stability of its components.

Recommendations for the project's current state

1. **Strengthen the confidentiality of information.** It is recommended to review the authentication mechanisms, session management, and credential storage, as well as to protect API keys and prevent them from being exposed in the source code. It is also important to verify that the data sent to external services, such as the Groq API, does not include unnecessary sensitive information.

2. **Protect the gateway itself and the continuity of the backend.** It is recommended to add an input chain in nftables with a drop policy, restrict SSH by key to the users' VLAN only, rotate the secrets exposed during development, and migrate the PostgreSQL database to a plan with no expiration, since the free database expired on October 27, 2026 and its current renewal is temporary.
3. **Complete and document the security tests.** Controlled pentesting tests should be expanded to verify the detection and blocking of threats, the effectiveness of the firewall rules, and the isolation between VLANs. It is likewise necessary to document the results of

each test, the alerts generated, and the corrections applied in order to have verifiable evidence of the system's effectiveness.

4. **Implement up-to-date backups.** It is recommended to create backups of the gateway configuration, the firewall rules, the VLANs, the PostgreSQL database, and the other essential components. Restoration tests should also be performed to verify that the system can recover from hardware failures, microSD card corruption, or data loss.
5. **Improve monitoring and threat response.** It is advisable to continue tuning the detection rules to reduce false positives and improve the identification of suspicious activity. Blocking actions must be verified through controlled tests, ensuring that security measures do not interrupt the devices' legitimate communications.
6. **Verify the integrity of logs and configurations.** It is recommended to restrict access permissions to the security logs, keep a history of administrative changes, and establish mechanisms to detect unauthorized modifications. This would facilitate incident investigation and prevent the evidence of a threat from being altered or deleted.
7. **Keep the technological components up to date.** It is important to periodically review the versions of the operating system, security tools, software dependencies, and libraries used. Before updating, compatibility between components and the operation of existing configurations must be checked to avoid new interruptions.
8. **Continue improving the user experience.** The interface should present alerts through clear messages, understandable risk levels, and practical recommendations. It is also recommended to verify that the actions available to the user are safe and that the information displayed matches the actual state of the network.
9. **Strengthen session management and the access layer.** It is recommended to move the rate limiter to shared storage (Redis), replace the token in the SSE connection URL with a single-use ticket, implement explicit JWT revocation, and verify an own domain in the email service.

Recommendations for future development

1. **Develop a scalable management and monitoring system.** In the future, it is recommended to evaluate incorporating a SIEM that makes it possible to centralize, correlate, and analyze security events from different installations. This would facilitate the identification of attack patterns and allow QUILLNET's monitoring capabilities to be expanded.
2. **Expand the automation of incident response.** Quillnet already automatically isolates, by MAC address, devices that generate 3 real-block alerts in 5 minutes; it is proposed to expand that automation with more decision criteria, apply additional security rules, and restore normal conditions when appropriate. These actions must have authorization criteria and verification mechanisms to avoid incorrect blocks.
3. **Evaluate the implementation of QUILLNET as a service managed by an internet provider.** One line of development consists of studying how to deploy and manage the system from the infrastructure of an internet service provider (ISP). To do so, it will be necessary to analyze the costs, compatibility with different network equipment, secure remote administration, and the maintenance of multiple installations.
4. **Improve the ability to adapt to different IoT devices.** It is recommended to expand testing with IP cameras, smart TVs, voice assistants, and other devices from different manufacturers. This would make it possible to identify differences in behavior, adjust the segmentation policies, and verify that the protection works in different home environments.
5. **Optimize performance and availability.** In future versions, the hardware's processing and memory capacity, the response times to alerts, and the impact of monitoring on connectivity should be evaluated. It is also worth studying recovery alternatives and configurations that reduce dependence on a single point of failure.
6. **Expand validation through Purple Team exercises.** It is recommended to establish a periodic cycle of attack simulation, detection, results analysis, and defense improvement. This process will make it possible to verify the system's behavior against new threats and to maintain a continuous evaluation of its protection capabilities.

7. **Evaluate alternatives to reduce dependence on external services.** It is proposed to study options for alert interpretation that allow the system to remain functional when an external artificial intelligence service is unavailable. Any alternative must be evaluated according to its accuracy, cost, privacy, and resource consumption. As a first step, the current version already classifies the attack type with fixed rules before resorting to the AI and displays fallback text by severity if the AI does not respond.
8. **Carry out a commercial feasibility study.** Before considering large-scale distribution, it is recommended to analyze hardware costs, maintenance, updates, technical support, and the operation of the service. Possible partnerships with internet providers should also be identified, and an affordable cost model for Costa Rican households should be established. The model projected in Chapter III (₡3,500 per month and recovery of the investment in 14 months) must be validated with real operating cost data.

In general terms, it is recommended to maintain a continuous, documented, and evidence-based development process. The evolution of QUILLNET should prioritize information protection, system reliability, ease of use, and the validation of its security mechanisms, so that each new feature responds to an identified need and contributes to the main objective of protecting domestic IoT devices.

Bibliography

3CORESec. (n.d.). *tmNIDS (testmynids.org)*. 3CORESec/testmynids.org repository on GitHub. https://testmynids.org

Akamai. (2026). *Akamai Guardicore Segmentation*. https://www.akamai.com/resources/product-brief/akamai-segmentation

Bitdefender. (2025, March 27). *Securing your smart home: Step-by-step network segmentation*. https://bitdefender.in/securing-your-smart-home-step-by-step-network-segmentation/

Cisco. (2024). *Cisco IoT threat defense: Securing the IoT at scale*. https://blogs.cisco.com/security/cisco-iot-threat-defense-securing-the-iot-at-scale

Cisco. (2026, January 30). *What is network segmentation?* https://www.cisco.com/site/us/en/learn/topics/security/what-is-network-segmentation.html

Cloudflare. (2026). *Cloudflare 2025 Q4 DDoS threat report*. https://blog.cloudflare.com/ddos-threat-report-2025-q4/

ColorTokens. (2025, May 8). *ColorTokens + Nozomi Networks partnership for OT/IoT security*. https://colortokens.com/news/nozomi-iot-ot-security-vendors/

Cybersecurity and Infrastructure Security Agency (CISA). (n.d.). *Known exploited vulnerabilities catalog*. https://www.cisa.gov/known-exploited-vulnerabilities-catalog

Droms, R. (1997). *Dynamic Host Configuration Protocol* (RFC 2131). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc2131

Federal Bureau of Investigation, Internet Crime Complaint Center (IC3). (2025, June 5). *Home Internet Connected Devices Facilitate Criminal Activity* (Alert I-060525-PSA). https://www.ic3.gov/PSA/2025/PSA250605

Forescout Research – Vedere Labs. (2025). *The riskiest devices of 2025*.

https://www.forescout.com/research-labs/the-riskiest-devices-of-2025/

Fortinet. (2024). *FortiNAC — Network access control*.

https://www.fortinet.com/products/network-access-control

Fortinet. (n.d.). *¿Mejores prácticas de seguridad del IoT? Cómo proteger los dispositivos del IoT [IoT security best practices? How to protect IoT devices]*.

https://www.fortinet.com/lat/resources/cyberglossary/iot-best-practices

Groq. (n.d.). *GroqCloud documentation*. https://console.groq.com/docs

Help Net Security. (2024, September 19). *FBI forced Flax Typhoon to abandon its botnet*.

https://www.helpnetsecurity.com/2024/09/19/flax-typhoon-botnet-disrupted/

Institute of Electrical and Electronics Engineers. (2022). *IEEE standard for local and metropolitan area networks—Bridges and bridged networks* (IEEE Std 802.1Q-2022). IEEE.

IoT Analytics. (2025, October 30). *State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally*. IoT Business News.

https://iotbusinessnews.com/?p=44499

IoT For All. (2024, December 2). *10 tips to secure your IoT devices from hackers*.

https://www.iotforall.com/10-tips-to-secure-your-iot-devices-from-hackers

Ip_Admin. (2024, August 28). *10 common IoT hacks and how to defend against them*. Device Authority. https://deviceauthority.com/10-common-iot-hacks/

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7519

Jouly, R. (n.d.). *Reducción de la superficie de ataque en entornos IoT [Reducing the attack surface in IoT environments]*. ManageEngine.

https://www.manageengine.com/latam/blog/general/reduccion-de-la-superficie-de-ataque-en-entornos-ti.html

JumpCloud. (2025). *IoT device vulnerability report*. https://jumpcloud.com

Kelley, S. (n.d.). *Dnsmasq: Network services for small networks*.

https://thekelleys.org.uk/dnsmasq/doc.html

McRae. (n.d.). *Comprendiendo los pilares clave de la tecnología IoT [Understanding the key pillars of IoT technology]*. SpinTel. https://www.spintel.net.au/blog/pillars-of-iot

Meta. (2024). *Llama 3.3 model card*. https://www.llama.com

Microsoft Azure. (n.d.). *¿Qué es IoT? [What is IoT?]* https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-iot

MITRE. (2024). *MITRE ATT&CK framework*. https://attack.mitre.org/

MITRE Caldera. (2024). *Caldera: Automated adversary emulation*. https://caldera.mitre.org/

Netfilter Project. (n.d.). *nftables wiki*. https://wiki.nftables.org/wiki-nftables/index.php/What_is_nftables%3F#What_is_nftables?

Netgear / Bitdefender. (2025). *2025 IoT threat report*. https://www.netgear.com/hub/network/security/iot-threat-report-25/

National Institute of Standards and Technology (NIST). (2020). *Zero Trust Architecture (SP 800-207)*. https://www.nist.gov/publications/zero-trust-architecture

National Institute of Standards and Technology (NIST). (2025). *Digital Identity Guidelines (SP 800-63-4)*.

National Institute of Standards and Technology (NIST). (2025). *Authentication and Authenticator Management (SP 800-63B-4)*.

MDN Web Docs. (2025). *API*. https://developer.mozilla.org

MDN Web Docs. (2025). *What is a web server?* https://developer.mozilla.org

Open Web Application Security Project (OWASP). (2025). *Password Storage Cheat Sheet*. https://owasp.org

Open Web Application Security Project (OWASP). (2025). *OWASP Top 10: Cryptographic Failures*. https://owasp.org

Open Information Security Foundation. (2025). *Suricata User Guide*. https://suricata.io

Pallets Projects. (2025). *Handling Application Errors*. https://flask.palletsprojects.com

Palo Alto Networks, Unit 42. (2020). *2020 Unit 42 IoT Threat Report*. https://unit42.paloaltonetworks.com/iot-threat-report-2020/

Palo Alto Networks. (2024). *Enterprise IoT security*.

https://www.paloaltonetworks.com/network-security/device-security

Percival, C., & Josefsson, S. (2016). *The scrypt password-based key derivation function* (RFC 7914). Internet Engineering Task Force.

https://www.rfc-editor.org/info/rfc7914/

PostgreSQL Global Development Group. (2025). *PostgreSQL documentation*.

https://www.postgresql.org/docs/

Proofpoint. (n.d.). *Emerging Threats Open ruleset*. https://rules.emergingthreats.net

Raspberry Pi Ltd. (2025). *Raspberry Pi 4 Model B: Product brief and documentation*.

https://www.raspberrypi.com/documentation/

Rekhter, Y., Moskowitz, B., Karrenberg, D., de Groot, G. J., & Lear, E. (1996). *Address allocation for private internets* (RFC 1918). Internet Engineering Task Force www.rfc-editor.org/info/rfc1918/

SentinelOne. (2025, October 23). *Los 10 principales riesgos de seguridad del IoT y cómo mitigarlos [The 10 main IoT security risks and how to mitigate them]*.https://www.sentinelone.com/es/cybersecurity-101/data-and-ai/iot-security-risks/

SQLite Project. (2025). *About SQLite*. https://sqlite.org

Srisuresh, P., & Egevang, K. (2001). *Traditional IP network address translator (Traditional NAT)* (RFC 3022). Internet Engineering Task Force. www.rfc-editor.org/rfc/rfc3022

Transforma Insights. (2026). *IoT connections forecast 2024-2035*.

https://transformainsights.com/research/forecast/highlights

Trend Micro. (2023, May 2). *Mirai botnet exploits TP-Link router vulnerability: What to know and how to protect your device*.

https://news.trendmicro.com/2023/05/02/mirai-botnet-exploits-tp-link-router-vulnerability

WHATWG. (n.d.). *HTML living standard: Server-sent events*.

https://html.spec.whatwg.org/multipage/server-sent-events.html

WifiTalents. (2026). *IoT default credentials and update statistics*. https://wifitalents.com

Zscaler. (2024). *IoT Security Solutions*.

https://www.zscaler.com/products-and-solutions/iot-security

¿Qué es una superficie de ataque? [What is an attack surface?] (n.d.). IBM.

https://www.ibm.com/es-es/think/topics/attack-surface

Annexes

Annex 1. Additional statistics

The average risk of connected devices in the ten highest-risk countries rose from 6.53 in 2024 to 9.1 in 2025 (Forescout, 2025). In September 2024, the FBI, the NSA, and the Cyber National Mission Force revealed a botnet of more than 260,000 devices (routers, IP cameras, video recorders, and network-attached storage devices) controlled by the Chinese company Integrity Technology Group (Help Net Security, 2024).

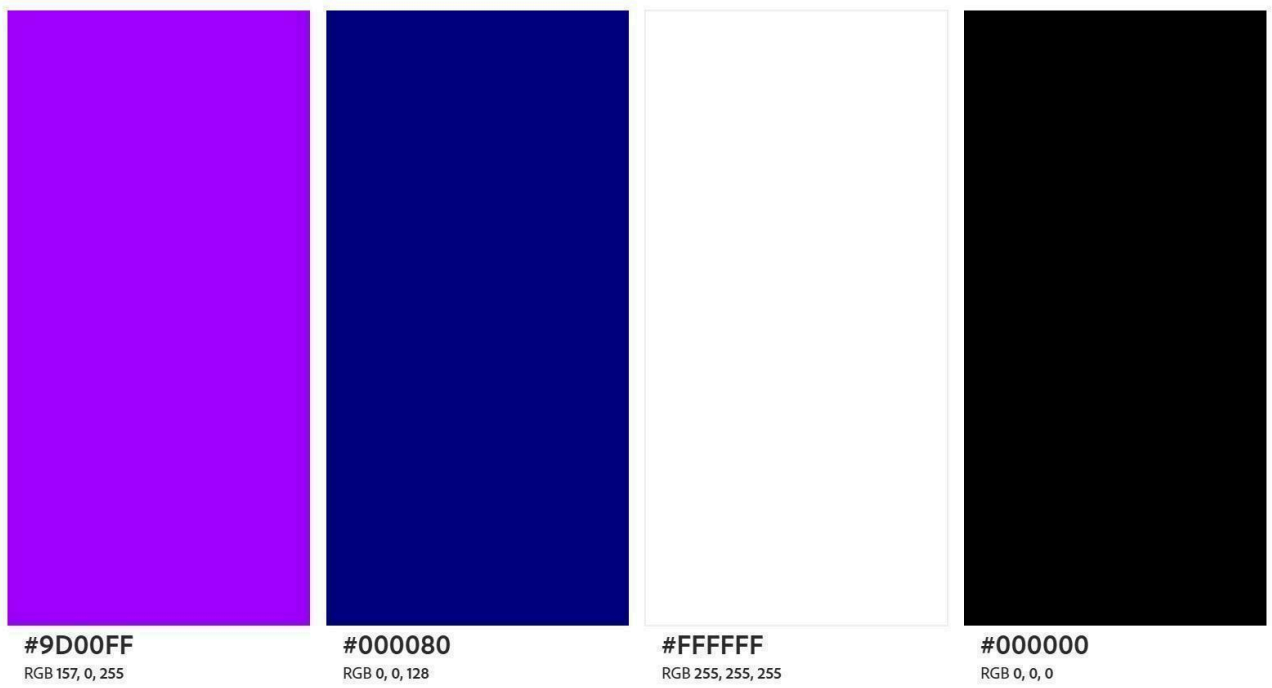
Annex 2. Abbreviated technical glossary

IoT (Internet of Things), CVE (Common Vulnerabilities and Exposures), MQTT (Message Queuing Telemetry Transport), VLAN (Virtual Local Area Network), IDS (Intrusion Detection System), SIEM (Security Information and Event Management), ISP (Internet Service Provider), ZTA (Zero Trust Architecture), 2FA (Two-Factor Authentication), DHCP (Dynamic Host Configuration Protocol), DNS (Domain Name System), ET Open (Emerging Threats Open, a public Suricata rule set), GPL (a rule set inherited from Snort/Sourcefire for Suricata), HMAC (Hash-based Message Authentication Code), IPS (Intrusion Prevention System), JA3 (fingerprint of the client's TLS handshake), JWT (JSON Web Token), KEV (Known Exploited Vulnerabilities, a CISA catalog), MAC (Media Access Control), NFQUEUE (nftables queue that delivers packets to Suricata in inline mode), PSK (Pre-Shared Key), PWA (Progressive Web App), scrypt (a memory-hard key derivation function), SNI (Server Name Indication), SSE (Server-Sent Events), SSID (Service Set Identifier), TLS (Transport Layer Security), VAPID (Voluntary Application Server Identification), Web Push (web push notification standard).

Annex 3. Project technology stack

Technology	Main use	Use in the project
Suricata IPS	Intrusion detection system	Analyzes network traffic, blocks attacks, and generates alerts when it detects something suspicious
Kali Linux	Attack simulation and pentesting	Responsible for simulating the attacks against each device
Groq API	Programming interface for connecting artificial intelligence models	Translates the network's technical alerts into plain language for the user
Raspberry Pi	Compact single-board computer	It is the physical hardware on which the entire system runs
Raspberry Pi OS Lite	Official Raspberry Pi operating system, based on Debian	Gateway operating system; it replaced OpenWRT, which did not allow Suricata to be installed
Python Flask	Microframework for building applications and APIs	It is the API that connects to the web application to show data to the user
PostgreSQL	Open-source relational database	Cloud database (Render) with data separated by household
HTML/CSS/JavaScript	Fundamental technologies of the World Wide Web	They make up the web application's frontend to show data to the user
nftables and dnsmasq	Linux firewall and DHCP/DNS service	Zero Trust policy (forward drop) and one DHCP/DNS scope per VLAN
Render	Cloud deployment platform	Hosts the Flask backend and the PostgreSQL database
Resend	Email service	Password recovery, two-step verification code (login and unlock), and backup of high alerts
Web Push (VAPID) and PWA	Notifications and installable application	Notification of alerts and new devices

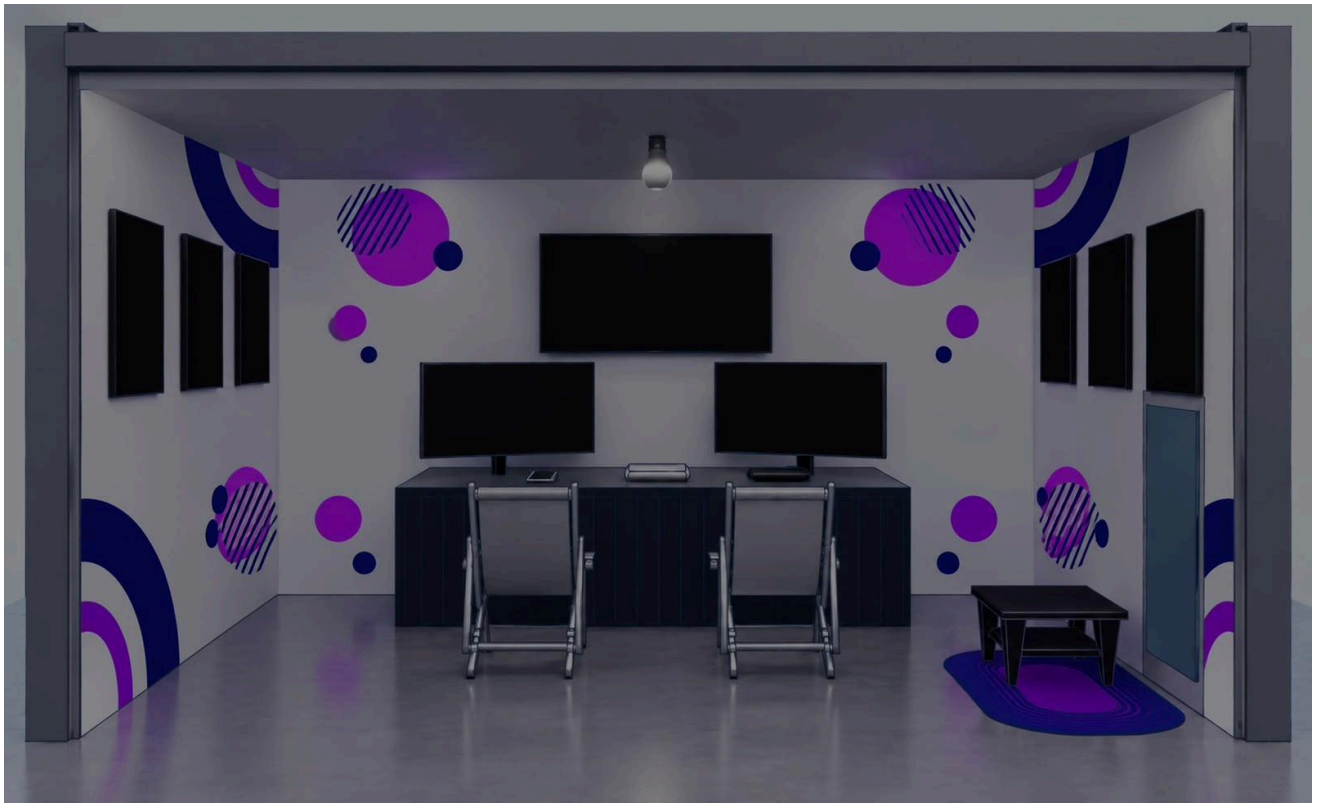
Annex 4. Color Palette



Annex 5. Logo



Annex 6. Stand design.



Annex 7. View of a VLAN from the phone in dark mode.



Annex 8. View of the alerts from the phone in light mode.



Annex 9. View of a VLAN from the phone in light mode.



Annex 10. View of the status dashboard from the phone in dark mode.



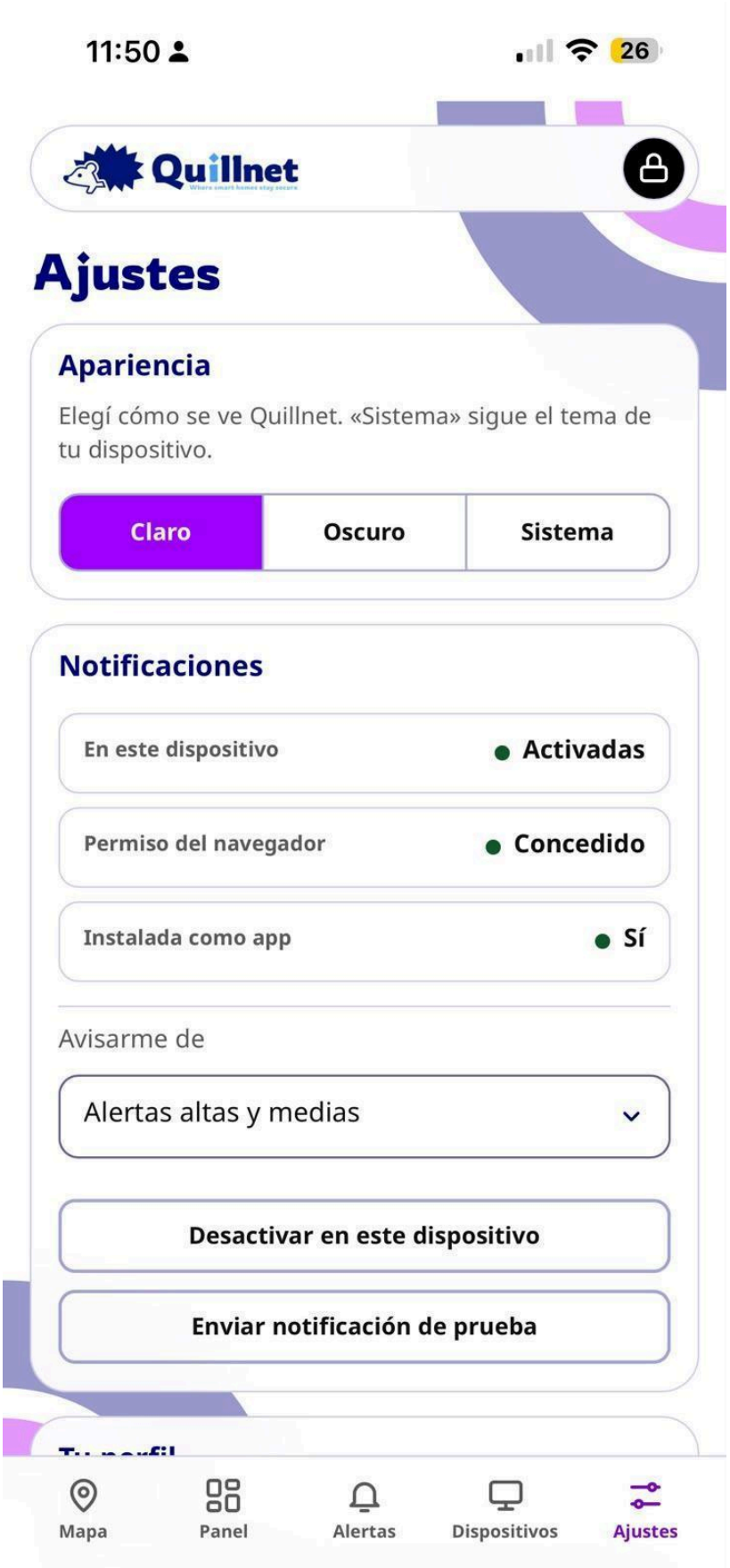
Annex 11. View of the status dashboard from the phone in light mode.



Annex 12. View of the settings from the phone in dark mode.



Annex 13. View of the settings from the phone in light mode.



Annex 14. View of the alerts from the phone in dark mode.



Annex 15. Instant alerts.



Annex 16. Alert and device description.



Annex 17. Sample of an attack in Kali Linux (sqlmap)



Annex 18. Sample of Suricata logs.

